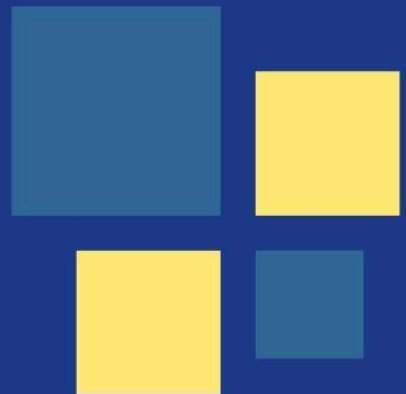


Rangkuman Belajar Pemrograman Dasar Python 2026



Hak Cipta

Rangkuman Belajar Pemrograman Dasar Python 2026

Oleh: [Kaka Ridwan](#)

Email: halo.kakaridwan@gmail.com

Copyright © 2026 oleh [Kaka Ridwan](#). Hak Cipta Dilindungi Undang-Undang.

Selesai disusun: 10 Juli 2026.

Revisi terakhir: 20 September 2026.

PEMBERITAHUAN PENTING:

E-book ini dibagikan secara gratis dan diperbolehkan untuk diunduh, disimpan, serta disebarluaskan kepada orang lain.

- **Wajib mencantumkan sumber:** Saat membagikan atau menggunakan materi dari e-book ini, tetap cantumkan nama penulis dan judul e-book.
- **Dilarang mengklaim sebagai karya sendiri:** Dilarang menyalin atau menggunakan sebagian maupun seluruh isi e-book lalu mengakuinya sebagai karya sendiri.
- **Dilarang menjual kembali:** File asli e-book ini tidak boleh dijual atau didistribusikan untuk memperoleh keuntungan.

UNDUH & PRODUK LAINNYA

E-book ini dapat diunduh secara gratis melalui:

<https://lynk.id/kaka.rdown>

Pada halaman tersebut, Anda juga dapat menemukan produk digital lainnya.

Terima kasih telah menghargai karya penulis.

Disclaimer

Dengan membaca e-book ini, Anda memahami dan menyetujui beberapa hal berikut:

- **Proses Belajar & Pemahaman:** Penulis ([Kaka Ridwan](#)) menyusun e-book ini sebagai bagian dari proses belajar. Oleh karena itu, e-book ini tidak dimaksudkan sebagai referensi yang sepenuhnya menggantikan dokumentasi resmi. Cara dan kecepatan memahami materi dapat berbeda pada setiap individu.
- **Bantuan AI dalam Penyusunan:** Dalam proses penyusunan e-book ini, penulis menggunakan bantuan AI untuk membantu merapikan dan menyusun kata-kata serta penjelasan agar lebih mudah dipahami. Materi yang dibahas, susunan pembelajaran, dan konsep yang dimasukkan ditentukan oleh penulis. Contoh-contoh kode yang terdapat dalam e-book ini juga dipraktikkan oleh penulis selama proses penyusunannya.
- **Risiko Error:** Contoh kode dalam e-book ini disediakan untuk tujuan pembelajaran. Meskipun telah dipraktikkan dan diperiksa selama proses penyusunan, tetap dimungkinkan terdapat kesalahan atau perbedaan hasil akibat perubahan versi Python, perangkat, lingkungan pengembangan, maupun faktor teknis lainnya. Penulis tidak bertanggung jawab atas kerusakan atau kerugian yang mungkin timbul akibat penggunaan materi dalam e-book ini.
- **Perubahan Teknologi:** Materi dalam e-book ini disusun berdasarkan teknologi dan praktik pemrograman Python yang dipelajari pada saat penulisan. Python, pustaka, lingkungan pengembangan, dan berbagai teknologi terkait dapat mengalami perubahan di masa mendatang. Oleh karena itu, sebagian materi atau contoh dalam e-book ini mungkin perlu disesuaikan dengan perkembangan teknologi dan versi Python di kemudian hari.
- **Sumber Referensi:** Beberapa penjelasan dalam e-book ini merujuk pada dokumentasi dan sumber belajar yang tersedia secara publik. Sumber yang digunakan dicantumkan sebagai referensi atau catatan kaki pada bagian yang relevan.

Kata Pengantar

E-book ini sengaja saya beri judul "Rangkuman Belajar" karena dua alasan jujur.

Pertama, kata "Rangkuman" terinspirasi dari tugas legendaris yang pasti sering kita dengar sejak SD sampai bangku kuliah: "Rangkum bab sekian!" Lewat kata ini, saya ingin berterus terang sejak awal bahwa materi di dalam e-book ini disusun sebagai intisari dari berbagai konsep dasar pemrograman Python. Setiap materi dipilih untuk membantu pembaca memahami hal-hal penting dan penerapannya melalui contoh sederhana.

Kedua, kata "Belajar" disematkan karena saya yang menulis buku ini pun masih berada dalam proses yang sama dengan Anda. Kita masih sama-sama belajar, mencoba, menemukan kesalahan, dan terus berkembang.

E-book ini saya susun sebagai teman belajar untuk memahami dasar-dasar pemrograman Python. Daripada harus berpindah-pindah dari berbagai sumber untuk menemukan penjelasan mengenai konsep yang sedang dipelajari, saya mencoba menyatukannya dalam satu rangkuman yang dapat digunakan sebagai bahan belajar maupun referensi saat ingin mengingat kembali materi yang pernah dipelajari.

Saya berharap e-book sederhana ini dapat membantu Anda memahami dasar-dasar pemrograman Python dan menjadi langkah awal untuk mempelajari hal-hal yang lebih lanjut.

Selamat belajar dan semoga bermanfaat!

Daftar Isi

Hak Cipta.....	i
Disclaimer.....	ii
Kata Pengantar.....	iii
Daftar Isi.....	iv
Bab 1: Pendahuluan.....	1
Apa itu Interpreter?.....	1
Instalasi Python.....	2
Editor Kode.....	3
Ekstensi Python di VS Code.....	3
Membuat Program Python Pertama.....	5
Membuat Komentar di Python.....	6
Komentar Satu Baris.....	6
Komentar Multi-Baris.....	6
Aturan Penulisan Dasar Python.....	7
Huruf Besar dan Kecil (Case Sensitive).....	7
Indentasi.....	7
Penggunaan Titik Koma (;).....	7
Bab 2: Variabel dan Tipe Data.....	8
Variabel.....	8
Cara Deklarasi Variabel.....	8
Aturan Penamaan Variabel.....	8
Menampilkan Nilai Variabel.....	9
Gaya Penulisan Variabel.....	10
Tipe Data.....	12
Tipe Data Utama.....	12
Tipe Data Koleksi (Struktur Data).....	14

Mendapatkan Tipe Data dengan type().....	17
Type Hint (Typing).....	18
Konversi Tipe Data.....	19
Bab 3: Operator.....	21
Operator Aritmatika.....	21
Operator + (Penjumlahan).....	21
Operator - (Pengurangan).....	21
Operator * (Perkalian).....	22
Operator / (Pembagian).....	22
Operator // (Pembagian Bulat).....	22
Operator % (Modulus).....	22
Operator ** (Pangkat).....	22
Operator Perbandingan.....	22
Operator == (Sama Dengan).....	22
Operator != (Tidak Sama Dengan).....	22
Operator < (Lebih Kecil).....	23
Operator > (Lebih Besar).....	23
Operator <= (Lebih Kecil atau Sama Dengan).....	23
Operator >= (Lebih Besar atau Sama Dengan).....	23
Operator Logika.....	23
Operator and.....	23
Operator or.....	24
Operator not.....	24
Operator Bitwise.....	24
Operator & (AND).....	24
Operator ^ (XOR).....	24

Operator (OR).....	24
Operator ~ (NOT).....	25
Operator << (Geser Kiri).....	25
Operator >> (Geser Kanan).....	25
Operator Penugasan.....	25
Operator =.....	25
Operator +=.....	25
Operator -=.....	26
Operator *=.....	26
Operator /=.....	26
Operator //=.....	26
Operator %=.....	26
Operator **=.....	27
Operator &=.....	27
Operator =.....	27
Operator ^=.....	27
Operator <<=.....	27
Operator >>=.....	28
Operator Identitas.....	28
Operator is.....	28
Operator is not.....	28
Operator Keanggotaan.....	29
Operator in.....	29
Operator not in.....	29
Bab 4: Percabangan.....	30
Penggunaan if, elif, dan else.....	30

if.....	30
else.....	31
elif.....	31
Ternary.....	32
Percabangan match-case.....	33
Fitur di match-case.....	34
Percabangan Bersarang (Nested If).....	36
Penjelasan Alur Program.....	37
Catatan.....	38
Bab 5: Perulangan.....	39
Perulangan for.....	39
Perulangan pada String.....	39
Perulangan pada List.....	39
Perulangan pada Dictionary.....	40
Perulangan dengan Fungsi range().....	41
Perulangan while.....	42
Penggunaan continue, break, dan else.....	43
continue.....	43
break.....	44
else.....	45
Perulangan Bersarang.....	47
Bab 6: Fungsi.....	49
Parameter.....	51
Multi-Parameter.....	51
Parameter Default.....	52
Parameter Fleksibel.....	53

Fungsi dengan Nilai Kembalian (Return Function).....	56
Scope Variabel.....	57
Variabel Lokal.....	57
Variabel Global.....	58
Lambda (Anonymous Function).....	58
Lambda dengan Banyak Parameter.....	59
Fungsi Bawaan (Built-in Functions).....	60
Fungsi Input / Output (I/O).....	60
Fungsi Iterasi dan Koleksi.....	61
Fungsi Matematika Dasar.....	63
Bab 7: Pemrograman Berorientasi Objek.....	66
Class dan Objek.....	66
Class.....	66
Objek.....	67
Komponen Utama dalam Class.....	67
Konstruktor.....	67
Properti (Atribut).....	68
Method.....	69
Penggunaan self.....	70
Membuat dan Mengubah Properti.....	71
Mengubah Nilai Properti.....	71
Menambahkan Properti Baru.....	71
Menghapus Properti.....	72
Class Attribute dan Instance Attribute.....	72
Instance Attribute.....	72
Class Attribute.....	73

Pilar-Pilar OOP.....	74
Encapsulation (Enkapsulasi).....	74
Inheritance (Pewarisan).....	76
Polymorphism (Polimorfisme).....	80
Abstraction (Abstraksi).....	81
Bab 8: Penanganan Error.....	83
Jenis-Jenis Error.....	83
Syntax Error (Kesalahan Penulisan).....	83
Runtime Error (Kesalahan Saat Program Berjalan).....	84
Logical Error (Kesalahan Logika).....	86
Penanganan Error dengan try dan except.....	86
Penanganan Error Tertentu pada except.....	88
Penggunaan else pada Penanganan Error.....	89
Penggunaan finally pada Penanganan Error.....	90
Penggunaan raise.....	92
Penggunaan raise di Dalam try.....	92
Penggunaan raise di Dalam except.....	93
Membuat Exception Kustom.....	95
Bab 9: Modul dan Package.....	97
Modul.....	97
Membuat dan Mengimpor Modul.....	97
Modul Bawaan (Built-in Modules).....	99
Modul math.....	99
Modul datetime.....	100
Modul random.....	100
Modul os.....	100

Package.....	101
Membuat dan Mengimpor Package.....	101
Mengimpor Package.....	102
PyPI.....	103
Bab 10: Pengolahan File.....	105
Mode Pengolahan File.....	105
Mode r.....	105
Mode w.....	105
Mode a.....	105
Mode x.....	106
Mode rb.....	106
Mode wb.....	106
Mode ab.....	106
Mode r+.....	106
Membuka dan Menutup File.....	106
Menulis File.....	107
Menulis Beberapa Baris.....	109
Menulis Beberapa Baris dengan writelines().....	109
Menambahkan Isi File.....	110
Menambahkan Data dengan write().....	110
Menambahkan Beberapa Baris dengan writelines().....	111
Membaca File.....	112
Membaca Seluruh Isi File dengan read().....	112
Membaca Satu Baris dengan readline().....	113
Membaca Seluruh Baris dengan readlines().....	114
Penggunaan with open().....	115

Bab 1: Pendahuluan

Python adalah bahasa pemrograman tingkat tinggi yang mudah dibaca dan dipahami, dengan sintaks yang sederhana dan mendekati bahasa manusia. Python bersifat umum (*general-purpose*)¹ dan dapat digunakan untuk berbagai kebutuhan seperti pengembangan web, analisis data, dan kecerdasan buatan.

Karena kemudahannya, Python sering menjadi bahasa pertama yang dipelajari oleh pemula dalam dunia pemrograman.²

Python termasuk bahasa pemrograman yang menggunakan *interpreter*, yaitu kode program dapat dijalankan secara langsung tanpa harus melalui proses kompilasi terlebih dahulu. Hal ini memudahkan proses pengembangan dan pengujian program karena hasil dapat langsung terlihat.

Python merupakan bahasa pemrograman *open-source*³ yang dikelola oleh [Python Software Foundation](https://www.python.org/), sehingga dapat digunakan secara bebas untuk keperluan belajar maupun komersial.⁴

Apa itu Interpreter?

Interpreter adalah program yang menjalankan kode sumber secara langsung, baris demi baris, tanpa mengubahnya menjadi file khusus terlebih dahulu. Dengan interpreter, kesalahan pada kode dapat langsung diketahui saat program dijalankan, misalnya jika ada baris kode yang salah, program akan langsung berhenti di bagian tersebut.

Berbeda dengan bahasa yang menggunakan *compiler*⁵, *interpreter* tidak memerlukan proses penerjemahan awal, sehingga lebih praktis digunakan untuk belajar dan pengembangan awal.

¹ General-purpose adalah istilah untuk bahasa pemrograman yang dapat digunakan untuk berbagai jenis kebutuhan, bukan hanya satu tujuan tertentu.

² <https://www.oracle.com/asean/developer/what-is-python-for-developers/>

³ Open-source adalah perangkat lunak yang kode sumbernya terbuka untuk umum sehingga dapat digunakan, dipelajari, dimodifikasi, dan didistribusikan secara bebas.

⁴ <https://docs.python.org/3/license.html>

⁵ Compiler adalah program yang menerjemahkan seluruh kode sumber menjadi file program (biasanya file executable) sebelum dijalankan.

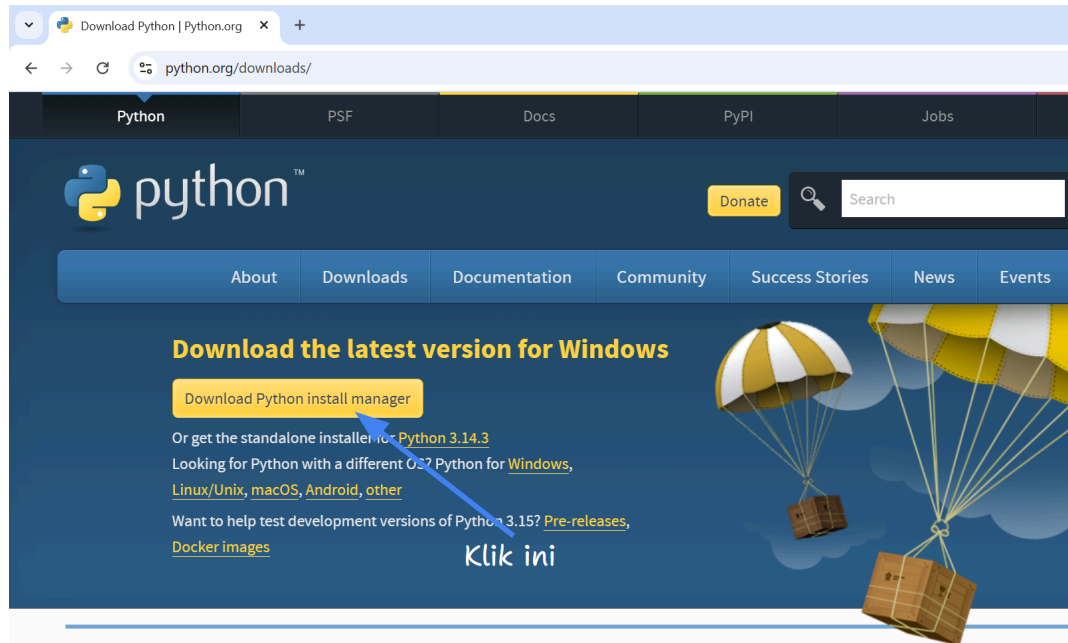
Instalasi Python

Berikut langkah langkah untuk menginstal Python:

1. Download Python Install Manager

Kunjungi situs resmi Python:

<https://www.python.org/downloads/>



2. Jalankan Installer dan Klik "Install Python"

Buka file yang telah diunduh dan klik tombol "Install Python".



3. Jika muncul pesan "Add commands directory to your PATH now? [y/N]", maka ketik y, lalu tekan tombol enter.

```
*****
The global shortcuts directory is not configured.

Configuring this enables commands like python3.14.exe to run from your
terminal, but is not needed for the python or py commands (for example, py
-V:3.14).

We can add the directory (C:\Users\          \AppData\Local\Python\bin) to
PATH now, but you will need to restart your terminal to use it. The entry will
be removed if you run py uninstall --purge, or else you can remove it manually
when uninstalling Python.
Add commands directory to your PATH now? [y/N] |
```

4. Setelah instalasi selesai, buka Command Prompt (CMD) atau Terminal, lalu ketik `python --version`.

```
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\          >python --version
Python 3.14.3
```

Jika berhasil, akan muncul versi Python yang terpasang.

Editor Kode

Untuk menulis dan menjalankan program Python, kita dapat menggunakan editor kode atau *Integrated Development Environment* (IDE)⁶.

Beberapa editor yang dapat digunakan antara lain:

- [Visual Studio Code \(VS Code\)](#)
- [PyCharm](#)
- [Notepad++](#)

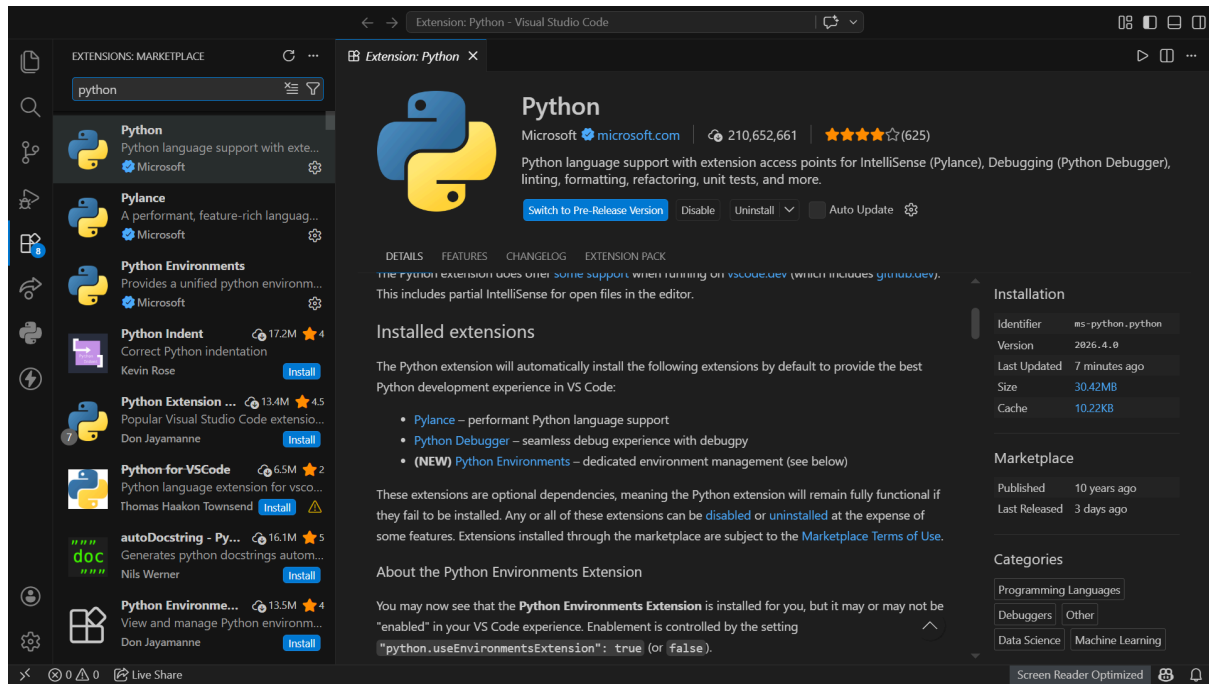
Dalam ebook ini, akan digunakan [Visual Studio Code \(VS Code\)](#) karena ringan, mudah digunakan, dan cocok untuk pemula.

Ekstensi Python di VS Code

Agar dapat menulis dan menjalankan program Python dengan lebih mudah di Visual Studio Code (VS Code), diperlukan ekstensi Python.

Ekstensi Python yang digunakan adalah [Python yang dikembangkan oleh Microsoft](#).

⁶ Integrated Development Environment adalah aplikasi yang menyediakan berbagai alat dalam satu tempat untuk menulis, menjalankan, dan mengelola kode program.



Ekstensi ini berfungsi untuk menambahkan fitur penting seperti:

- Penyorotan sintaks (*syntax highlighting*)⁷
- Saran kode (*auto-complete*)⁸
- Menjalankan dan *debugging program*⁹

Saat ekstensi ini diinstal, biasanya akan otomatis memasang ekstensi tambahan seperti:

- [Pylance](#) (untuk saran kode)
- [Python Debugger](#)

⁷ Penyorotan sintaks (*syntax highlighting*) adalah fitur yang memberi warna berbeda pada elemen kode agar lebih mudah dibaca dan dipahami.

⁸ Saran kode (*auto-complete*) adalah fitur yang menampilkan dan melengkapi kode secara otomatis saat kita mengetik untuk mempercepat penulisan program.

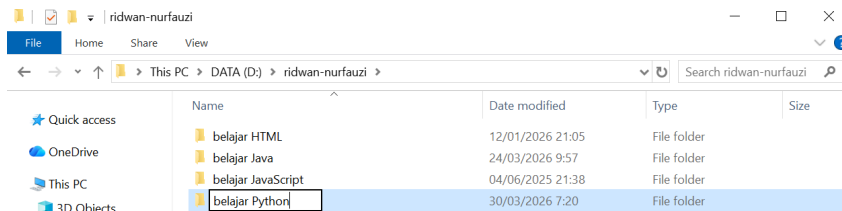
⁹ Debugging program adalah proses mencari dan memperbaiki kesalahan (*bug*) dalam kode agar program berjalan dengan benar.

Membuat Program Python Pertama

Berikut langkah-langkah untuk membuat program Python pertama:

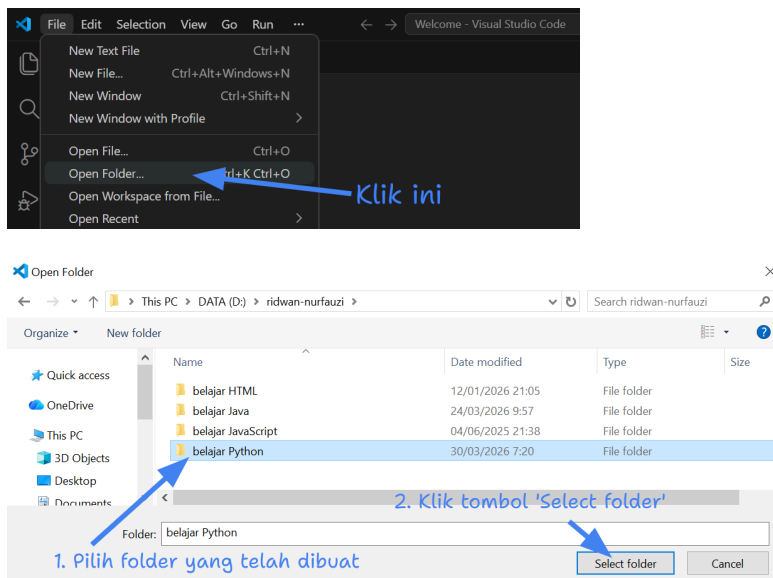
1. Buat Folder Baru

Digunakan untuk menyimpan file program.



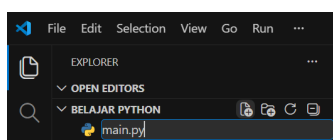
2. Buka Folder Tersebut Di Visual Studio Code

Pilih menu *Open Folder*, lalu buka folder yang telah dibuat.



3. Buat File Baru

Setelah folder terbuka di Visual Studio Code, buat file baru dengan nama `main.py`.

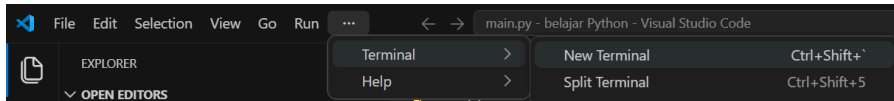


4. Isi Kode Python

```
print("Hello World!")
```

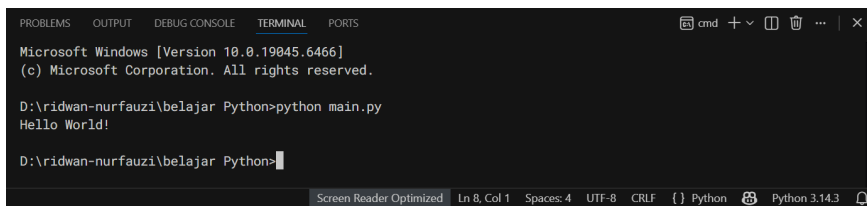
5. Buka Terminal

Pilih menu *New Terminal*, atau menggunakan *shortcut* `ctrl + shift + ``.



6. Jalankan Python

Ketik `python main.py` lalu tekan tombol `enter` di terminal, maka hasilnya akan muncul di terminal.



Membuat Komentar di Python

Komentar adalah bagian dari kode yang tidak dijalankan oleh Python dan digunakan untuk memberikan penjelasan pada program.

Komentar Satu Baris

Komentar satu baris diawali dengan tanda `#`.

```
# Ini contoh komentar satu baris
print("Hello World!")
```

Komentar Multi-Baris

Python tidak memiliki tanda khusus untuk komentar multi-baris seperti `/* ... */` pada Java, namun sering menggunakan tanda kutip tiga (`'''` atau `"""`) sebagai alternatif.

```
"""
Ini adalah komentar
lebih dari satu baris
"""
print("Hello World!")
```

Catatan: Contoh komentar multi-baris di atas sebenarnya adalah [string \(teks\)](#) multi-baris yang tidak digunakan, jadi program mengabaikannya.

Aturan Penulisan Dasar Python

Dalam Python, terdapat beberapa aturan dasar yang perlu diperhatikan:

Huruf Besar dan Kecil (*Case Sensitive*)

Python membedakan huruf besar dan huruf kecil.

```
nama = "Udin"  
Nama = "Edo"  
print(nama) # Udin  
print(Nama) # Edo
```

Variabel `nama` dan `Nama` dianggap sebagai dua variabel yang berbeda.

Indentasi

Python menggunakan indentasi (spasi di awal baris) untuk menentukan blok kode.

```
if True:  
    print("Ini dijalankan")
```

Tanpa indentasi yang benar, program akan error.

Catatan:

- Indentasi biasanya menggunakan spasi (*space*)
- Bisa juga menggunakan tab, tetapi sebaiknya konsisten
- Di Visual Studio Code, indentasi biasanya sudah otomatis saat menekan Enter setelah tanda `:`

Penggunaan Titik Koma (;)

Python tidak mewajibkan penggunaan tanda titik koma (;) di akhir baris.

```
print("Hello")
```

Namun, tanda `;` dapat digunakan untuk menulis beberapa perintah dalam satu baris:

```
print("Hello"); print("World")
```

Meskipun bisa, cara ini jarang digunakan karena dapat mengurangi keterbacaan kode.

Bab 2: Variabel dan Tipe Data

Variabel

Variabel adalah tempat untuk menyimpan suatu nilai di dalam program. Setiap variabel memiliki tipe data yang menentukan jenis nilai yang disimpan, seperti angka, teks, atau nilai logika.

Cara Deklarasi Variabel

Dalam Python, variabel dibuat dengan cara langsung memberikan nilai ke sebuah nama variabel tanpa perlu menentukan tipe data secara eksplisit.

Contoh:

```
nama = "Ridwan"
```

Berdasarkan kode di atas:

- `nama` adalah nama variabelnya
- `"Ridwan"` adalah nilai yang disimpan di dalam variabel tersebut.

Aturan Penamaan Variabel

Dalam Python, nama variabel harus mengikuti aturan tertentu agar dapat digunakan dengan benar.

Aturan Dasar

- Nama variabel tidak boleh diawali dengan angka
- Nama variabel hanya boleh diawali dengan:
 - Huruf (a–z atau A–Z)
 - Garis bawah (`_`)
- Setelah karakter pertama, nama variabel boleh terdiri dari:
 - Huruf
 - Angka
 - Garis bawah (`_`)

- Tidak boleh menggunakan karakter lain seperti @, -, #, dan sebagainya

Jika aturan ini dilanggar, maka program akan menghasilkan error.

Contoh yang Benar

```
nama = "Ridwan"  
_nama = "Ridwan"  
umur1 = 20
```

Contoh yang Salah

```
1nama = "Ridwan" # diawali angka  
@nama = "Ridwan" # diawali simbol
```

Menampilkan Nilai Variabel

Menggunakan `print()` Langsung

Cara ini merupakan cara paling sederhana untuk menampilkan nilai variabel. Output yang dihasilkan hanya berupa nilai dari variabel tersebut tanpa tambahan teks apa pun.

```
nama = "Ridwan"  
print(nama)
```

Menggunakan F-String

F-String adalah cara modern untuk menampilkan variabel yang digabung dengan teks. Metode ini lebih mudah dibaca dan direkomendasikan karena lebih rapi serta fleksibel. F-String diperkenalkan melalui [Python Enhancement Proposal \(PEP\) 498](#) pada Python 3.6.

```
nama = "Ridwan"  
print(f>Nama: {nama})
```

Menggunakan `.format()`

Metode `.format()` digunakan untuk menggabungkan teks dengan variabel sebelum adanya [F-String](#).

```
nama = "Ridwan"  
umur = 20  
print("Nama saya adalah {}. Umur saya {} tahun.".format(nama, umur))
```

Menggunakan Tanda Koma (,)

Cara ini digunakan untuk menampilkan teks dan variabel secara bersamaan dengan memisahkannya menggunakan tanda koma. Secara otomatis, Python akan menambahkan spasi di antara teks dan nilai variabel.

```
nama = "Ridwan"  
print("Nama:", nama)
```

Menggunakan Operator Modulo (%)

Operator modulo (%) merupakan cara lama untuk memformat dan menampilkan variabel. Saat ini cara ini sudah jarang digunakan, tetapi masih perlu untuk diketahui.

Pada metode ini, simbol seperti %d digunakan sebagai *format specifier*, yaitu penanda tipe data yang akan ditampilkan.

```
tahun = 2026  
print("Sekarang adalah tahun %d." % tahun)
```

Pada contoh di atas, %d digunakan untuk menampilkan nilai *integer* (angka bulat).

Format Specifier yang umum digunakan:

- %s : String (teks)
- %d : Integer (angka bulat)
- %f : Float (desimal)
contoh: %.2f (2 angka di belakang koma)
- %x : Hexadecimal

Gaya Penulisan Variabel

Dalam Python, nama variabel tidak boleh menggunakan spasi karena akan menyebabkan error. Oleh karena itu, penulisan nama variabel biasanya menggunakan gaya penamaan tertentu (*naming convention*) agar kode lebih rapi dan mudah dibaca.

snake_case

snake_case adalah gaya penulisan yang menggunakan huruf kecil dan tanda underscore (`_`) sebagai pemisah antar kata. Gaya ini paling disarankan untuk penulisan nama variabel dan fungsi di Python.

```
nama_pengguna = "kakaridwan"  
umur_pengguna = 20
```

camelCase

camelCase adalah gaya penulisan yang menggunakan huruf kecil pada kata pertama, kemudian setiap kata berikutnya diawali huruf besar tanpa menggunakan pemisah.

```
namaPengguna = "kakaridwan"  
umurPengguna = 20
```

UPPERCASE_SNAKE_CASE

Python tidak memiliki variabel konstan (*constant*)¹⁰ secara khusus. Namun, biasanya nilai yang dianggap konstan ditulis menggunakan UPPERCASE_SNAKE_CASE, yaitu semua huruf kapital dengan underscore (`_`) sebagai pemisah antar kata.

```
PI = 3.14  
MAX_USERS = 1000
```

PascalCase

PascalCase adalah gaya penulisan yang membuat setiap awal kata menggunakan huruf kapital tanpa pemisah. Gaya ini biasanya digunakan untuk penamaan class.

```
UserProfile  
ProductItem
```

¹⁰ Variabel konstan adalah variabel yang nilainya dianggap tetap dan tidak seharusnya diubah.

Tipe Data

Tipe data adalah jenis nilai yang disimpan dalam variabel. Dalam Python, tipe data variabel akan ditentukan secara otomatis berdasarkan nilai yang diberikan.

Tipe Data Utama

Integer (`int`)

Integer adalah tipe data untuk bilangan bulat positif maupun negatif.

```
angka1 = 4
angka2 = -5
```

Float (`float`)

Float adalah tipe data untuk bilangan desimal yang menggunakan tanda titik (.) sebagai pemisah.

```
pi = 3.14
suhu = -0.5
```

String (`str`)

String adalah tipe data yang digunakan untuk menyimpan teks atau kumpulan karakter. String ditulis menggunakan tanda kutip tunggal ('...') atau tanda kutip ganda ("...").

```
nama = "Ridwan"
pesan = 'Hello World'
```

String juga dapat ditulis dalam bentuk multi-baris menggunakan tanda kutip tiga (''' atau """).

```
teks = """
Ini adalah contoh
string multi-baris
"""
```

Penulisan ini mirip seperti contoh [komentar multi-baris](#) yang telah dibahas sebelumnya.

Karakter Khusus pada String (*Escape Character*)

Escape character adalah karakter khusus dalam string yang diawali dengan tanda backslash (\) untuk memberikan arti tertentu yang tidak bisa ditulis secara langsung.

Contohnya:

```
print("Baris pertama\nBaris kedua")
```

Output:

```
Baris pertama
Baris kedua
```

Beberapa *escape character* yang sering digunakan:

- `\n` : membuat baris baru
- `\t` : membuat jarak indentasi (*tab*)
- `\'` : menyisipkan tanda petik tunggal
- `\"` : menyisipkan tanda petik ganda
- `\\` : menampilkan tanda backslash (\) itu sendiri

Contohnya:

```
print("Halo\tPython") # output: Halo   Python
print('It\'s Python') # output: It's Python
print("C:\\Program Files\\Python") # output: C:\Program Files\Python
```

Boolean (`bool`)

Boolean adalah tipe data logika yang hanya memiliki dua nilai, yaitu `True` untuk benar atau `False` untuk salah.

```
is_login = True
is_admin = False
```

Dalam Python, `True` dan `False` harus diawali huruf kapital.

Complex (complex)

Complex adalah tipe data untuk bilangan kompleks. Dalam matematika, bilangan imajiner biasanya menggunakan huruf `i`, sedangkan di Python menggunakan huruf `j`.

```
bilangan1 = 4j
bilangan2 = 2 + 1j
```

NoneType (None)

None adalah nilai khusus yang digunakan untuk menyatakan bahwa suatu variabel tidak memiliki nilai.

```
x = None
```

Tipe Data Koleksi (Struktur Data)

Selain tipe data dasar, Python juga memiliki tipe data koleksi (*collection*) yang digunakan untuk menyimpan banyak nilai dalam satu variabel. Tipe data koleksi termasuk struktur data, yaitu cara untuk menyimpan dan mengelola data agar lebih teratur dan mudah digunakan.

List (list)

List adalah tipe data yang digunakan untuk menyimpan kumpulan nilai. Nilai di dalam list dapat memiliki tipe data yang berbeda dan isi list dapat diubah (*mutable*).

List ditulis menggunakan tanda kurung siku (`[` dan `]`).

```
data = [1, 1, 2, "tiga"]
buah = ["apel", "jeruk", "mangga"]
```

Indeks (Index)

```
buah = ["apel", "jeruk", "mangga"]
       0      1      2
```

Setiap nilai di dalam list memiliki indeks (*index*). Indeks adalah urutan posisi nilai di dalam list yang dimulai dari `0`.

```
buah = ["apel", "jeruk", "mangga"]

print(buah[0]) # output: apel
print(buah[1]) # output: jeruk
print(buah[2]) # output: mangga
```

Salah satu nilai di dalam list dapat diakses maupun diubah menggunakan format `nama_variabel[index]`.

```
buah = ["apel", "jeruk", "mangga"]
buah[1] = "anggur"
print(buah)
# output: ['apel', 'anggur', 'mangga']
```

Slice

Selain menggunakan indeks tunggal, list juga dapat mengambil beberapa nilai sekaligus menggunakan slice dengan format:

```
nama_variabel[start:stop:step]
```

Keterangan:

- `start` : indeks awal pengambilan data
- `stop` : indeks akhir pengambilan data (tidak ikut diambil)
- `step` : jarak atau langkah pengambilan data

```
numbers = [1,2,3,4,5,6,7,8,9,10]
```

Mengambil data mulai dari indeks tertentu hingga akhir list:

```
print(numbers[3:]) # output: [4, 5, 6, 7, 8, 9, 10]
```

Mengambil data dari awal list hingga sebelum indeks tertentu:

```
print(numbers[:5]) # output: [1, 2, 3, 4, 5]
```

Mengambil data dari indeks awal hingga sebelum indeks akhir:

```
print(numbers[2:7]) # output: [3, 4, 5, 6, 7]
```

Mengambil data dengan jarak tertentu:

```
print(numbers[::2]) # output: [1, 3, 5, 7, 9]
```

Menggunakan start, stop, dan step secara bersamaan:

```
print(numbers[1:8:2]) # output: [2, 4, 6, 8]
```

Python juga mendukung indeks negatif untuk mengakses data dari belakang list:

```
print(numbers[-1]) # output: 10
print(numbers[-3:]) # output: [8, 9, 10]
```

Slice tidak hanya dapat digunakan pada list, tetapi juga dapat digunakan pada [tuple](#) dan [string](#).

Tuple (tuple)

Tuple mirip seperti list, tetapi isi datanya tidak dapat diubah (*immutable*) setelah dibuat.

Tuple ditulis menggunakan tanda kurung biasa ((dan)).

```
data = (1, 1, 2, "tiga")
buah = ("apel", "jeruk", "mangga")
```

Tuple juga memiliki indeks (*index*) seperti list.

```
buah = ("apel", "jeruk", "mangga")

print(buah[0]) # output: apel
print(buah[1]) # output: jeruk
```

Nilai pada tuple dapat diakses menggunakan `nama_variabel[index]`, tetapi tidak dapat diubah.

```
buah = ("apel", "jeruk", "mangga")
buah[1] = "anggur" # error
```

Dictionary (dict)

Dictionary digunakan untuk menyimpan data dalam bentuk pasangan kunci dan nilai (*key-value*). Setiap key harus unik dan tidak boleh ada key yang sama dalam satu dictionary.

Dictionary ditulis menggunakan tanda kurung kurawal ({ dan }).

```
data = {
    'nama': 'Kaka Ridwan',
    'umur': 19,
}
```

Salah satu data pada dictionary dapat diakses menggunakan `data[key]`.

```
print(data) # output: {'nama': 'Kaka Ridwan', 'umur': 19}
print(data['nama']) # output: Kaka Ridwan
print(data['umur']) # output: 19
```

Value pada dictionary juga dapat diubah menggunakan `data[key]`.

```
data['umur'] = 20
print(data['umur']) # output: 20
```

Menggunakan `.get()`

Method `.get(key)` digunakan untuk mengambil nilai berdasarkan `key`.

```
print(data.get('umur')) # output: 20
```

Menggunakan `.setdefault()`

Method `.setdefault(key, value)` digunakan untuk menambahkan data baru jika `key` tersebut belum ada.

```
data.setdefault('alamat', 'Bandung')
print(data) # output: {'nama': 'Kaka Ridwan', 'umur': 20, 'alamat': 'Bandung'}
print(data.get('alamat')) # output: Bandung
```

Set (`set`)

Set adalah tipe data yang berisi kumpulan nilai unik dan tidak terurut. Nilai di dalam set tidak boleh duplikat.

Set ditulis menggunakan tanda kurung kurawal (`{` dan `}`).

```
data = {1, 2, 3}
```

Jika terdapat nilai yang sama, maka Python akan otomatis menghapus nilai duplikat tersebut.

```
data = {1, 2, 3, 3, 4, 5}
print(data) # output: {1, 2, 3, 4, 5}
```

Mendapatkan Tipe Data dengan `type()`

Python menyediakan fungsi `type()` untuk mengetahui tipe data dari suatu nilai atau variabel.

```
nama = "Ridwan"
umur = 20
tinggi = 170.5

print(type(nama)) # output: <class 'str'>
print(type(umur)) # output: <class 'int'>
print(type(tinggi)) # output: <class 'float'>
```

`type()` juga dapat digunakan langsung pada nilai tanpa variabel.

```
print(type(True)) # output: <class 'bool'>
```

Type Hint (Typing)

Python mendukung *type hint* (typing), yaitu penulisan tipe data sebagai petunjuk pada variabel atau [fungsi](#) agar kode lebih mudah dibaca, dipahami, serta membantu editor kode dalam memberikan saran kode.

Type hint tidak wajib digunakan, tetapi dapat membantu membuat kode lebih mudah dibaca dan dipahami.

```
nama: str = "Ridwan"
umur: int = 20
tinggi: float = 170.5
```

Pada contoh di atas:

- `str` adalah petunjuk bahwa variabel nama berisi [string](#)
- `int` adalah petunjuk bahwa variabel umur berisi [integer](#)
- `float` adalah petunjuk bahwa variabel tinggi berisi [desimal \(float\)](#)

Type hint hanya bersifat petunjuk dan tidak membatasi tipe data secara ketat saat program dijalankan.

```
umur: int = "dua puluh"
print(umur) # output: dua puluh
```

Meskipun `umur` diberi petunjuk `int`, program tetap berjalan dan tidak menghasilkan error.

Type hint pertama kali diperkenalkan pada Python 3.5 melalui [PEP 484](#), namun saat itu penggunaannya masih berfokus pada fungsi. Pada Python 3.6, dukungan *type hint* untuk variabel mulai diperkenalkan melalui *variable annotations* pada [PEP 526](#).

Konversi Tipe Data

Konversi tipe data adalah proses mengubah suatu nilai menjadi tipe data lain. Dalam Python, konversi tipe data dapat terjadi secara otomatis ([konversi implisit](#)) maupun dilakukan secara manual ([konversi eksplisit](#)).

Konversi Implisit

Konversi implisit adalah konversi tipe data yang dilakukan otomatis oleh Python.

Contohnya:

```
angka = 10
desimal = 2.5
hasil = angka + desimal
print(type(angka)) # output: <class 'int'>
print(type(desimal)) # output: <class 'float'>
print(type(hasil)) # output: <class 'float'>
```

Pada contoh di atas, nilai [integer](#) otomatis diubah menjadi [float](#) oleh Python.

Konversi Eksplisit

Konversi explicit adalah konversi tipe data yang dilakukan secara manual menggunakan [fungsi](#) tertentu.

Tidak semua nilai dapat dikonversi ke tipe data tertentu. Jika format nilai tidak sesuai, maka akan terjadi error.

`str()`

Fungsi `str()` digunakan untuk mengubah nilai menjadi [string](#).

```
print(str(20)) # output: '20'
print(str(False)) # output: 'False'
```

`int()`

Fungsi `int()` digunakan untuk mengubah nilai menjadi [integer](#) (angka bulat).

```
print(int(3.14)) # output: 3
print(int(True)) # output: 1
```

`float()`

Fungsi `float()` digunakan untuk mengubah nilai menjadi [float](#) (desimal).

```
print(float(10)) # output: 10.0
print(float("3.14")) # output: 3.14
print(float(True)) # output: 1.0
```

bool()

Fungsi `bool()` digunakan untuk mengubah nilai menjadi [boolean](#).

```
print(bool(1)) # output: True
print(bool(0)) # output: False
```

list()

Fungsi `list()` digunakan untuk mengubah nilai menjadi [list](#).

```
print(list("halo")) # output: ['h', 'a', 'l', 'o']
print(list((1, 2, 3))) # output: [1, 2, 3]
print(list({1, 2, 3})) # output: [1, 2, 3]
```

tuple()

Fungsi `tuple()` digunakan untuk mengubah nilai menjadi [tuple](#).

```
print(tuple("halo")) # output: ('h', 'a', 'l', 'o')
print(tuple({1, 2, 3})) # output: (1, 2, 3)
```

set()

Fungsi `set()` digunakan untuk mengubah nilai menjadi [set](#).

```
print(set([1, 2, 3])) # output: {1, 2, 3}
```

Set juga dapat digunakan untuk menghapus nilai duplikat secara otomatis.

```
print(set([1, 1, 2, 2, 3])) # output: {1, 2, 3}
```

dict()

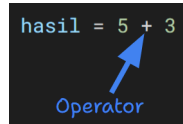
Fungsi `dict()` digunakan untuk membuat atau mengubah data menjadi [dictionary](#).

```
print(dict(nama="Ridwan", umur=20)) # output: {'nama': 'Ridwan', 'umur': 20}

print(dict([
    ('a', 1),
    ('b', 2)
]))
# output: {'a': 1, 'b': 2}
```

Bab 3: Operator

Operator adalah simbol yang digunakan untuk melakukan operasi terhadap nilai atau variabel.



```
hasil = 5 + 3
```

Operator

Pada kode di atas, tanda `+` adalah operator penjumlahan.

Operator Aritmatika

Operator aritmatika digunakan untuk melakukan operasi matematika.

Operator `+` (Penjumlahan)

Operator `+` digunakan untuk menjumlahkan nilai.

```
print(5 + 3) # 8
```

Operator `+` juga dapat digunakan untuk menggabungkan string.

```
print("Halo" + " Dunia") # Halo Dunia
```

Namun, string tidak dapat langsung digabung dengan tipe data lain seperti `integer`.

```
umur = 20
print("Umur saya " + umur) # menghasilkan error
```

Agar dapat digabungkan, integer harus dikonversi menjadi string menggunakan `str()`.

```
umur = 20
print("Umur saya " + str(umur)) # Umur saya 20
```

Operator `-` (Pengurangan)

Operator `-` digunakan untuk mengurangi nilai.

```
print(10 - 3) # 7
```

Operator * (Perkalian)

Operator * digunakan untuk mengalikan nilai.

```
print(4 * 3) # 12
```

Operator / (Pembagian)

Operator / digunakan untuk pembagian dan hasilnya selalu berupa float.

```
print(10 / 2) # 5.0
```

Operator // (Pembagian Bulat)

Operator // digunakan untuk pembagian bulat.

```
print(10 // 3) # 3
```

Operator % (Modulus)

Operator % digunakan untuk mengambil sisa hasil bagi.

```
print(10 % 3) # 1
```

Operator ** (Pangkat)

Operator ** digunakan untuk perpangkatan.

```
print(2 ** 3) # 8
```

Operator Perbandingan

Operator perbandingan digunakan untuk membandingkan dua nilai dan menghasilkan nilai True atau False.

Operator == (Sama Dengan)

Operator == digunakan untuk memeriksa apakah dua nilai sama.

```
print(5 == 5) # True
```

Operator != (Tidak Sama Dengan)

Operator != digunakan untuk memeriksa apakah dua nilai tidak sama.

```
print(5 != 3) # True
```

Operator < (Lebih Kecil)

Operator < digunakan untuk memeriksa apakah nilai di sebelah kiri lebih kecil dari nilai di sebelah kanan.

```
print(3 < 5) # True
```

Operator > (Lebih Besar)

Operator > digunakan untuk memeriksa apakah nilai di sebelah kiri lebih besar dari nilai di sebelah kanan.

```
print(7 > 5) # True
```

Operator <= (Lebih Kecil atau Sama Dengan)

Operator <= digunakan untuk memeriksa apakah nilai di sebelah kiri lebih kecil atau sama dengan nilai di sebelah kanan.

```
print(5 <= 5) # True
```

Operator >= (Lebih Besar atau Sama Dengan)

Operator >= digunakan untuk memeriksa apakah nilai di sebelah kiri lebih besar atau sama dengan nilai di sebelah kanan.

```
print(7 >= 5) # True
```

Operator Logika

Operator logika digunakan untuk menggabungkan atau membalik kondisi [boolean](#) (True dan False).

Operator and

Operator and akan menghasilkan True jika kedua kondisi bernilai True.

```
print(True and True) # True  
print(True and False) # False
```

Operator or

Operator `or` akan menghasilkan `True` jika salah satu kondisi bernilai `True`.

```
print(True or False) # True
print(False or False) # False
```

Operator not

Operator `not` digunakan untuk membalik nilai `boolean`.

```
print(not True) # False
print(not False) # True
```

Operator Bitwise

Operator bitwise digunakan untuk operasi pada data biner (bit).

Agar lebih mudah memahami operator bitwise, sebaiknya pahami terlebih dahulu sistem bilangan biner dan desimal.

Operator & (AND)

Operator `&` akan menghasilkan bit `1` jika kedua bit bernilai `1`.

```
# 101 & 011 = 001
print(5 & 3) # 1
```

Operator ^ (XOR)

Operator `^` akan menghasilkan bit `1` jika kedua bit berbeda.

```
# 101 ^ 011 = 110
print(5 ^ 3) # 6
```

Operator | (OR)

Operator `|` akan menghasilkan bit `1` jika salah satu bit bernilai `1`.

```
# 101 | 011 = 111
print(5 | 3) # 7
```

Operator ~ (NOT)

Operator ~ digunakan untuk menghasilkan komplemen bit (bitwise NOT).

```
print(~5) # -6
```

Operator ini setara dengan:

```
~x == -(x + 1)
```

Operator << (Geser Kiri)

Operator << digunakan untuk menggeser bit ke kiri.

```
# 0101 menjadi 1010  
print(5 << 1) # 10
```

Operator >> (Geser Kanan)

Operator >> digunakan untuk menggeser bit ke kanan.

```
# 101 menjadi 010  
print(5 >> 1) # 2
```

Operator Penugasan

Operator penugasan digunakan untuk memberikan atau memperbarui nilai variabel.

Operator =

Operator = digunakan untuk memberikan nilai ke variabel.

```
x = 5  
print(x) # 5
```

Operator +=

Operator += digunakan untuk menjumlahkan nilai lalu menyimpannya kembali ke variabel.

```
x = 5  
x += 3  
print(x) # 8
```

Operator -=

Operator -= digunakan untuk mengurangi nilai lalu menyimpannya kembali ke variabel.

```
x = 5
x -= 2
print(x) # 3
```

Operator *=

Operator *= digunakan untuk mengalikan nilai lalu menyimpannya kembali ke variabel.

```
x = 5
x *= 2
print(x) # 10
```

Operator /=

Operator /= digunakan untuk membagi nilai lalu menyimpannya kembali ke variabel.

```
x = 10
x /= 2
print(x) # 5.0
```

Operator //=

Operator //= digunakan untuk pembagian bulat lalu menyimpannya kembali ke variabel.

```
x = 10
x //= 3
print(x) # 3
```

Operator %=

Operator %= digunakan untuk mengambil sisa hasil bagi lalu menyimpannya kembali ke variabel.

```
x = 10
x %= 3
print(x) # 1
```

Operator `**=`

Operator `**=` digunakan untuk perpangkatan lalu menyimpannya kembali ke variabel.

```
x = 2
x **= 3
print(x) # 8
```

Operator `&=`

Operator `&=` digunakan untuk operasi [bitwise AND](#) lalu menyimpannya kembali ke variabel.

```
x = 5
x &= 3
print(x) # 1
```

Operator `|=`

Operator `|=` digunakan untuk operasi [bitwise OR](#) lalu menyimpannya kembali ke variabel.

```
x = 5
x |= 3
print(x) # 7
```

Operator `^=`

Operator `^=` digunakan untuk operasi [bitwise XOR](#) lalu menyimpannya kembali ke variabel.

```
x = 5
x ^= 3
print(x) # 6
```

Operator `<<=`

Operator `<<=` digunakan untuk menggeser bit ke kiri lalu menyimpannya kembali ke variabel.

```
x = 5
x <<= 1
print(x) # 10
```

Operator `>>=`

Operator `>>=` digunakan untuk menggeser bit ke kanan lalu menyimpannya kembali ke variabel.

```
x = 5
x >>= 1
print(x) # 2
```

Operator Identitas

Operator identitas digunakan untuk memeriksa apakah dua variabel merujuk ke objek yang sama.

Operator `is`

Operator `is` digunakan untuk memeriksa apakah dua variabel merujuk ke objek yang sama.

```
x = [1, 2]
y = x
print(x is y) # True
```

Operator `is not`

Operator `is not` digunakan untuk memeriksa apakah dua variabel merujuk ke objek yang berbeda.

```
x = [1, 2]
y = [1, 2]
print(x is not y) # True
```

Operator Keanggotaan

Operator keanggotaan digunakan untuk memeriksa apakah suatu nilai terdapat di dalam objek seperti string, list, tuple, atau dictionary.

Operator `in`

Operator `in` digunakan untuk memeriksa apakah suatu nilai terdapat di dalam objek.

```
print("a" in "python") # False
print("p" in "python") # True
print(3 in [1, 2, 3]) # True
```

Operator `not in`

Operator `not in` digunakan untuk memeriksa apakah suatu nilai tidak terdapat di dalam objek.

```
print("z" not in "python") # True
print("p" not in "python") # False
print(5 not in [1, 2, 3]) # True
```

Bab 4: Percabangan

Percabangan adalah mekanisme dalam pemrograman yang memungkinkan program menjalankan kode tertentu berdasarkan kondisi bernilai True atau False.

Penggunaan `if`, `elif`, dan `else`

Dalam Python, percabangan umumnya menggunakan `if`, `elif`, dan `else`.

Ketiganya digunakan untuk menentukan kode mana yang akan dijalankan berdasarkan suatu kondisi.

`if`

`if` digunakan untuk menjalankan kode jika kondisi bernilai True.

Bentuk dasar:

```
if kondisi:
    kode
```

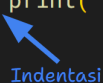
Contohnya:

```
nilai = 70

if nilai >= 60:
    print("Selamat, Anda lulus!")
```

Pastikan blok kodenya menggunakan `indentasi`.

```
if nilai >= 60:
    print("Selamat, Anda lulus!")
```



Output:

```
Selamat, Anda lulus!
```

Karena kondisi `nilai >= 60` bernilai True, maka blok kode nya dijalankan.

else

`else` digunakan untuk menjalankan kode ketika kondisi pada `if` atau pada kondisi lainnya bernilai `False`.

Bentuk dasar:

```
if kondisi:
    kode_jika_true
else:
    kode_jika_false
```

Contohnya:

```
nilai = 50

if nilai >= 60:
    print("Selamat, Anda lulus!")
else:
    print("Maaf, Anda tidak lulus.")
```

Output:

```
Maaf, Anda tidak lulus.
```

Karena kondisi `nilai >= 60` bernilai `False`, maka blok `else` dijalankan.

elif

`elif` adalah singkatan dari `else if`.

`elif` digunakan untuk memeriksa kondisi lain jika kondisi sebelumnya bernilai `False`.

Bentuk dasar:

```
if kondisi_1:
    kode_1
elif kondisi_2:
    kode_2
else:
    kode_lain
```

Contohnya:

```
nilai = 75

if nilai >= 85:
    print("A")
elif nilai >= 75:
    print("B")
elif nilai >= 60:
    print("C")
else:
    print("D")
```

Output:

```
B
```

Karena:

- `nilai >= 85` bernilai `False`
- `nilai >= 75` bernilai `True`

maka Python menjalankan blok kedua.

Ternary

Ternary digunakan untuk menyingkat percabangan `if-else` dengan langsung menghasilkan sebuah nilai berdasarkan kondisi.

Ternary cocok digunakan untuk percabangan sederhana yang hanya memiliki dua kemungkinan hasil.

Bentuk dasar:

```
nilai_jika_benar if kondisi else nilai_jika_salah
```

Artinya:

- jika kondisi bernilai `True`, maka menghasilkan `nilai_jika_benar`
- jika kondisi bernilai `False`, maka menghasilkan `nilai_jika_salah`

Contohnya:

```
nilai = 75

status = "Lulus" if nilai >= 60 else "Tidak Lulus"

print(f"Nilai: {nilai}, Status: {status}")
```

Output:

```
Nilai: 75, Status: Lulus
```

Karena kondisi `nilai >= 60` bernilai `True`, maka variabel status berisi `"Lulus"`.

Percabangan `match-case`

`match-case` adalah fitur Python yang diperkenalkan sejak Python 3.10 melalui [PEP 634](#), [PEP 635](#), dan [PEP 636](#).

`match-case` digunakan untuk memeriksa dan mencocokkan nilai atau struktur data.

Fitur ini mirip seperti `switch-case` pada bahasa pemrograman lain, tetapi lebih fleksibel karena dapat mencocokkan pola (*pattern*).

Bentuk dasar:

```
match variabel:
    case nilai_1:
        kode
    case nilai_2:
        kode
    case _:
        kode_default
```

Keterangan:

- `match` digunakan untuk memeriksa suatu nilai
- `case` digunakan untuk menentukan pola atau nilai yang dicocokkan
- `case _:` digunakan sebagai kondisi default jika tidak ada yang cocok

Contohnya:

```
buah = 'apel'
match buah:
    case 'apel':
        print('Buah ini adalah apel')
    case 'jeruk':
        print('Buah ini adalah jeruk')
    case _:
        print('Buah ini tidak diketahui')
```

Output:

```
Buah ini adalah apel
```

Karena nilai variabel `buah` adalah `"apel"`, maka Python menjalankan `case "apel"`.

Fitur di `match-case`

`match-case` di Python tidak hanya digunakan untuk mencocokkan teks atau angka saja, tetapi juga memiliki beberapa fitur tambahan.

Menggabungkan Banyak Kondisi

`match-case` bisa digunakan untuk memeriksa beberapa nilai sekaligus dalam satu `case` menggunakan operator pipe (`|`).

Contohnya:

```
huruf = "a"
match huruf:
    case "a" | "i" | "u" | "e" | "o":
        print("Huruf vokal")
    case _:
        print("Bukan huruf vokal")
```

Output:

```
Huruf vokal
```

Pada contoh di atas, jika nilai huruf cocok dengan salah satu huruf vokal, maka blok kode akan dijalankan.

Mencocokkan Struktur Data

`match-case` juga dapat membongkar isi `list` atau `tuple`.

Dengan fitur ini, Python dapat mencocokkan pola berdasarkan isi struktur data.

Contohnya:

```
koordinat = (0, 2)
match koordinat:
    case (0, 0):
        print("Titik pusat")
    case (0, y):
        print(f"Di sumbu Y, y = {y}")
    case (x, 0):
        print(f"Di sumbu X, x = {x}")
    case (x, y):
        print(f"x = {x}, y = {y}")
```

Output:

```
Di sumbu Y, y = 2
```

Pada contoh di atas `(0, y)` cocok dengan `(0, 2)` sehingga nilai `2` disimpan ke variabel `y`.

Menggunakan Kondisi Tambahan

Di dalam `case`, kita juga bisa menambahkan `if` untuk membuat kondisi yang lebih spesifik.

Fitur ini disebut *guard*.

Contohnya:

```
angka = 10
match angka:
    case x if x > 0:
        print("Bilangan positif")
    case x if x < 0:
        print("Bilangan negatif")
    case _:
        print("No!")
```

Output:

```
Bilangan positif
```

Pada contoh di atas:

```
case x if x > 0
```

berarti:

- simpan nilai ke variabel `x`
- lalu periksa apakah `x > 0`

Jika kondisi bernilai `True`, maka blok kode dijalankan.

Percabangan Bersarang (*Nested If*)

Percabangan bersarang adalah percabangan seperti `if` yang di dalamnya terdapat percabangan lainnya.

Biasanya percabangan bersarang digunakan ketika suatu kondisi harus diperiksa terlebih dahulu sebelum memeriksa kondisi berikutnya.

Bentuk dasar:

```
if kondisi_1:  
    if kondisi_2:  
        kode
```

Keterangan:

- kondisi pertama diperiksa terlebih dahulu
- jika bernilai `True`, maka kondisi kedua diperiksa
- jika kondisi kedua juga `True`, maka blok kode dijalankan

Contohnya:

```
username = "admin"
password = "1234"
if username == "admin":
    if password == "admin#1234":
        print("Login berhasil")
    else:
        print("Password salah")
else:
    print("Username tidak ditemukan")
```

Output:

```
Password salah
```

Penjelasan Alur Program

Pada contoh di atas, Python terlebih dahulu memeriksa kondisi:

```
username == "admin"
```

Karena bernilai `True`, maka Python masuk ke percabangan berikutnya:

```
password == "admin#1234"
```

Namun nilai variabel:

```
password = "1234"
```

tidak sama dengan:

```
"admin#1234"
```

sehingga kondisi bernilai `False`.

Karena kondisi bernilai `False`, maka Python menjalankan:

```
print("Password salah")
```

Jika kondisi:

```
username == "admin"
```

bernilai `False`, maka Python langsung menjalankan:

```
print("Username tidak ditemukan")
```

tanpa memeriksa password.

Catatan

Karena *nested if* memiliki beberapa tingkat percabangan, **indentasi** harus diperhatikan dengan benar.

```
if username == "admin":  
    if password == "admin#1234":  
        print("Login berhasil")  
    else:
```

Semakin dalam percabangan, maka indentasi akan semakin masuk ke dalam.

Bab 5: Perulangan

Perulangan adalah mekanisme pemrograman yang memungkinkan program menjalankan kode tertentu secara berulang berdasarkan kondisi atau jumlah tertentu.

Perulangan for

Perulangan `for` di Python adalah perintah untuk mengulang blok kode tertentu berdasarkan sebuah kumpulan data (*iterable*) seperti `list`, `tuple`, `string`, atau hasil dari fungsi `range()`.

Saat perulangan berjalan, `for` akan mengambil satu per satu elemen dari kumpulan data tersebut hingga seluruh elemen selesai diproses.

Bentuk dasar:

```
for item in items:
    kode
```

Perulangan pada String

Contoh perulangan `for` dengan `string`:

```
teks = "Halo"
for huruf in teks:
    print(huruf)
```

Output:

```
H
a
l
o
```

Karena `string` merupakan kumpulan karakter, perulangan akan mengambil setiap karakter satu per satu.

Perulangan pada List

Contoh perulangan `for` dengan `list` (hal yang sama juga dapat dilakukan pada `tuple`):

```
daftar_buah = ["apel", "jeruk", "mangga"]
for buah in daftar_buah:
    print(buah)
```

Output:

```
apel
jeruk
mangga
```

Pada contoh tersebut, variabel buah akan berisi setiap elemen dari list `daftar_buah` secara bergantian.

Perulangan pada Dictionary

Python juga dapat melakukan perulangan pada [dictionary](#).

Secara default, perulangan pada [dictionary](#) akan mengambil setiap `key` yang ada di dalam [dictionary](#).

Contohnya:

```
biodata = {
    'nama': 'Ridwan',
    'umur': 20,
    'status': 'mahasiswa',
}
for key in biodata:
    print(key)
```

Output:

```
nama
umur
status
```

Pada contoh tersebut, variabel `key` akan berisi setiap `key` dari [dictionary](#) `biodata` secara bergantian.

Jika ingin mengambil seluruh `value`, gunakan method `values()`.

Contohnya:

```
for value in biodata.values():
    print(value)
```

Output:

```
Ridwan
20
mahasiswa
```

Method `values()` menghasilkan kumpulan seluruh *value* yang ada pada `dictionary` sehingga dapat digunakan dalam perulangan.

Jika ingin mengambil *key* dan *value* sekaligus, gunakan method `items()`.

Contohnya:

```
for key, value in biodata.items():  
    print(f'{key}: {value}')
```

Output:

```
nama: Ridwan  
umur: 20  
status: mahasiswa
```

Method `items()` menghasilkan pasangan *key* dan *value* dari setiap elemen `dictionary`. Pada setiap perulangan, nilai *key* disimpan ke variabel *key* dan nilai *value* disimpan ke variabel *value*.

Perulangan dengan Fungsi `range()`

Selain menggunakan kumpulan data, Python juga menyediakan fungsi `range()` untuk menghasilkan deret angka. Fungsi ini sering digunakan ketika kita ingin melakukan perulangan sebanyak jumlah tertentu. Secara default, `range()` menghasilkan angka yang dimulai dari 0.

Contohnya:

```
for i in range(5):  
    print(f"Perulangan ke-{i}")
```

Output:

```
Perulangan ke-0  
Perulangan ke-1  
Perulangan ke-2  
Perulangan ke-3  
Perulangan ke-4
```

Pada contoh tersebut, `range(5)` menghasilkan deret angka 0, 1, 2, 3, dan 4, kemudian setiap angka disimpan ke dalam variabel *i* secara bergantian pada setiap perulangan.

Perulangan `while`

Perulangan `while` adalah perintah yang digunakan untuk mengulang suatu blok kode selama kondisi yang diberikan bernilai `True`.

Bentuk dasar:

```
while kondisi:  
    kode
```

Pada setiap perulangan, Python akan memeriksa kondisi yang diberikan. Jika kondisi bernilai `True`, blok kode di dalam `while` akan dijalankan. Jika kondisi bernilai `False`, perulangan akan berhenti.

Contoh:

```
angka = 1  
while angka <= 3:  
    print(f"Angka: {angka}")  
    angka += 1 # Baris ini penting agar nilai angka bertambah
```

Output:

```
Angka: 1  
Angka: 2  
Angka: 3
```

Cara kerja program di atas:

1. Variabel `angka` diberi nilai awal `1`.
2. Python memeriksa kondisi `angka <= 3`.
3. Karena `1 <= 3` bernilai `True`, maka `print()` dijalankan dan menampilkan `Angka: 1`.
4. Nilai `angka` kemudian ditambah `1` sehingga menjadi `2`.
5. Python kembali memeriksa kondisi `angka <= 3`.
6. Karena kondisi masih bernilai `True`, perulangan dijalankan kembali.
7. Proses ini terus berulang hingga nilai `angka` menjadi `4` atau kondisi `angka <= 3` sudah tidak terpenuhi.
8. Saat `angka` bernilai `4`, kondisi `angka <= 3` bernilai `False` sehingga perulangan berhenti.

Perhatikan bahwa pada contoh di atas terdapat baris:

```
angka += 1
```

Baris tersebut sangat penting karena digunakan untuk mengubah nilai `angka` pada setiap perulangan. Jika nilai `angka` tidak berubah, kondisi `angka <= 3` akan selalu bernilai `True` dan perulangan tidak akan pernah berhenti.

Contohnya:

```
angka = 1
while angka <= 3:
    print(f"Angka: {angka}")
```

Kode di atas akan terus berjalan tanpa henti karena nilai `angka` selalu bernilai `1`. Kondisi `angka <= 3` akan selalu bernilai `True`.

Perulangan yang tidak pernah berhenti seperti ini disebut *infinite loop* (perulangan tak terbatas).

Penggunaan `continue`, `break`, dan `else`

Dalam perulangan, Python menyediakan beberapa perintah khusus untuk mengatur jalannya perulangan, yaitu `continue`, `break`, dan `else`.

`continue`

Perintah `continue` digunakan untuk melewati sisa kode pada perulangan saat ini dan langsung melanjutkan ke perulangan berikutnya.

Perintah `continue` dapat digunakan pada perulangan `for` maupun `while`.

Contoh pada `for`:

```
for angka in range(5):
    if angka == 2:
        continue
    print(angka)
```

Output:

```
0
1
3
4
```

Pada contoh di atas, ketika nilai `angka` sama dengan `2`, perintah `continue` dijalankan sehingga `print(angka)` tidak dieksekusi. Perulangan langsung berlanjut ke iterasi¹¹ berikutnya.

Contoh pada `while`:

```
angka = 0
while angka < 5:
    angka += 1
    if angka == 3:
        continue
    print(angka)
```

Output:

```
1
2
4
5
```

Ketika nilai `angka` sama dengan `3`, perintah `continue` dijalankan sehingga `print(angka)` dilewati dan perulangan langsung berlanjut ke iterasi berikutnya.

break

Perintah `break` digunakan untuk menghentikan perulangan sebelum seluruh proses selesai.

Perintah `break` dapat digunakan pada perulangan `for` maupun `while`.

Contoh pada `for`:

```
for angka in range(5):
    if angka == 2:
        break
    print(angka)
```

Output:

```
0
1
```

Pada contoh di atas, ketika nilai `angka` sama dengan `2`, perintah `break` dijalankan sehingga perulangan langsung dihentikan.

¹¹ Iterasi adalah satu siklus di dalam perulangan.

Contoh pada `while`:

```
angka = 1
while angka <= 5:
    if angka == 4:
        break
    print(angka)
    angka += 1
```

Output:

```
1
2
3
```

Ketika nilai `angka` sama dengan 4, perintah `break` dijalankan sehingga perulangan langsung berhenti dan kode di dalam perulangan tidak dijalankan lagi.

`else`

Selain pada percabangan, `else` juga dapat digunakan pada perulangan `for` dan `while`.

Blok `else` akan dijalankan jika perulangan selesai secara normal, yaitu tanpa dihentikan oleh `break`.

Contoh pada `for`:

```
for angka in range(3):
    print(angka)
else:
    print("Perulangan selesai")
```

Output:

```
0
1
2
Perulangan selesai
```

Karena perulangan selesai secara normal, blok `else` dijalankan.

Contoh lain:

```
for angka in range(5):
    if angka == 2:
        break
    print(angka)
else:
    print("Perulangan selesai")
```

Output:

```
0
1
```

Pada contoh tersebut, blok `else` tidak dijalankan karena perulangan dihentikan oleh `break`.

Hal yang sama juga berlaku pada perulangan `while`.

Contoh:

```
angka = 1
while angka <= 3:
    print(angka)
    angka += 1
else:
    print("Perulangan selesai")
```

Output:

```
1
2
3
Perulangan selesai
```

Karena kondisi `while` menjadi `False` secara normal dan tidak ada `break`, blok `else` tetap dijalankan.

Perulangan Bersarang

Perulangan bersarang (*nested loop*) adalah perulangan yang berada di dalam perulangan lainnya.

Dengan kata lain, sebuah perulangan dapat memiliki perulangan lain di dalam blok kodenya.

Bentuk dasar:

```
for variabel1 in data1:
    for variabel2 in data2:
        kode
```

Ketika perulangan luar (*outer loop*) berjalan satu kali, perulangan dalam (*inner loop*) akan dijalankan hingga selesai.

Contoh:

```
for i in range(3):
    for j in range(2):
        print(f"i = {i}, j = {j}")
```

Output:

```
i = 0, j = 0
i = 0, j = 1
i = 1, j = 0
i = 1, j = 1
i = 2, j = 0
i = 2, j = 1
```

Cara Kerjanya:

1. `i` bernilai 0.
2. Perulangan `j` berjalan dari 0 hingga 1.
3. Setelah perulangan `j` selesai, nilai `i` berubah menjadi 1.
4. Perulangan `j` kembali berjalan dari 0 hingga 1.
5. Proses ini terus berulang hingga seluruh perulangan selesai.

Pada contoh di atas:

- Perulangan luar berjalan sebanyak 3 kali.
- Perulangan dalam berjalan sebanyak 2 kali setiap kali perulangan luar berjalan.
- Total perulangan yang terjadi adalah $3 \times 2 = 6$ kali.

Perulangan bersarang juga bisa digunakan untuk menampilkan pola bintang.

Contohnya:

```
for i in range(5):  
    for j in range(i + 1):  
        print("*", end=" ")  
    print() # Pindah ke baris baru setelah setiap baris bintang
```

Output:

```
*  
**  
***  
****  
*****
```

Dari contoh di atas, perulangan menampilkan jumlah bintang sesuai dengan urutan barisnya.

Bab 6: Fungsi

Fungsi (*function*) adalah sekumpulan kode yang dibungkus dalam satu blok dan diberi nama untuk menjalankan tugas tertentu.

Tujuan fungsi adalah mengelompokkan kode program agar dapat digunakan dan dipanggil berulang kali tanpa harus menulis kode yang sama berkali-kali. Dengan menggunakan fungsi, program menjadi lebih rapi, lebih mudah dibaca, dan lebih mudah dipelihara (*maintenance*).

Contoh tanpa fungsi:

```
nama = "Buku Tulis"
harga = 2000
print(f>Nama: {nama}")
print(f>Harga: {harga}")
print()

nama = "Penghapus"
harga = 4000
print(f>Nama: {nama}")
print(f>Harga: {harga}")
print()

nama = "Pulpen"
harga = 2000
print(f>Nama: {nama}")
print(f>Harga: {harga}")
print()
```

Pada contoh di atas, terdapat beberapa baris kode yang ditulis berulang kali. Jika jumlah data semakin banyak, maka kode akan menjadi semakin panjang dan sulit dikelola.

Untuk mengatasi hal tersebut, kita dapat menggunakan fungsi.

Contoh dengan fungsi:

```
def tampilkan_barang(nama, harga):  
    print(f>Nama: {nama}")  
    print(f>Harga: {harga}")  
    print()  
  
tampilkan_barang("Buku Tulis", 2000)  
tampilkan_barang("Penghapus", 4000)  
tampilkan_barang("Pulpen", 2000)
```

Pada contoh di atas, kode untuk menampilkan informasi barang cukup ditulis satu kali di dalam fungsi `tampilkan_barang()`. Setelah itu, fungsi dapat dipanggil berulang kali dengan data yang berbeda.

Dengan cara ini, kode menjadi lebih singkat, lebih rapi, dan lebih mudah dipahami.

Bentuk dasar:

```
def nama_fungsi(parameter):  
    kode
```

Keterangan:

- `def` digunakan untuk mendefinisikan atau membuat fungsi.
- `nama_fungsi` adalah nama fungsi yang akan digunakan untuk memanggil fungsi tersebut.
- `parameter` adalah data yang diterima oleh fungsi (opsional).
- `kode` adalah blok kode yang akan dijalankan ketika fungsi dipanggil.

Setelah fungsi dibuat, fungsi dapat dipanggil menggunakan nama fungsinya.

```
nama_fungsi(parameter)
```

Saat fungsi dipanggil, semua kode yang berada di dalam fungsi akan dijalankan.

Parameter

Parameter adalah variabel yang didefinisikan di dalam tanda kurung saat fungsi dibuat. Parameter berfungsi sebagai wadah untuk menerima data yang dikirimkan ke dalam fungsi.

Dengan adanya parameter, sebuah fungsi dapat digunakan berulang kali dengan data yang berbeda tanpa perlu mengubah kode di dalam fungsi tersebut.

Contoh:

```
def ucapkan_halo(nama):  
    print(f"Halo, {nama}! Selamat datang di dunia pemrograman.")  
  
ucapkan_halo("Udin")  
ucapkan_halo("Siti")  
ucapkan_halo("Edo")
```

Output:

```
Halo, Udin! Selamat datang di dunia pemrograman.  
Halo, Siti! Selamat datang di dunia pemrograman.  
Halo, Edo! Selamat datang di dunia pemrograman.
```

Pada contoh di atas, nama merupakan parameter. Setiap kali fungsi `ucapkan_halo()` dipanggil, nilai yang diberikan akan disimpan ke dalam parameter nama, kemudian digunakan oleh perintah `print()` yang berada di dalam fungsi.

Karena nilai parameter dapat berubah-ubah, fungsi yang sama dapat menghasilkan output yang berbeda sesuai data yang diterimanya.

Multi-Parameter

Sebuah fungsi juga dapat memiliki lebih dari satu parameter.

Contoh:

```
def tampilkan_barang(nama, harga):  
    print(f>Nama Barang: {nama}")  
    print(f>Harga Barang: Rp{harga}")  
  
tampilkan_barang("Buku Tulis", 2000)
```

Pada contoh tersebut, `nama` dan `harga` merupakan parameter yang digunakan untuk menerima dua data yang berbeda ke dalam fungsi.

Urutan data yang diberikan saat memanggil fungsi harus sesuai dengan urutan parameter yang didefinisikan pada fungsi. Pada contoh tersebut, nilai `"Buku Tulis"` akan disimpan ke parameter `nama`, sedangkan nilai `2000` akan disimpan ke parameter `harga`. Jika urutan data ditukar, maka hasil yang diperoleh juga akan berbeda.

Parameter Default

Parameter default adalah parameter yang memiliki nilai bawaan saat fungsi dibuat. Nilai bawaan ini akan digunakan secara otomatis jika saat pemanggilan fungsi tidak diberikan nilai untuk parameter tersebut.

Namun, jika nilai baru diberikan saat fungsi dipanggil, maka nilai bawaan tersebut akan digantikan oleh nilai yang baru diberikan.

Contoh:

```
def pesan_makan(menu, jumlah=1):  
    print(f"Pesanan: {menu} sebanyak {jumlah} porsi.")  
  
# Menggunakan nilai default  
pesan_makan("Nasi Goreng")  
  
# Mengganti nilai default  
pesan_makan("Mie Ayam", 3)
```

Output:

```
Pesanan: Nasi Goreng sebanyak 1 porsi.  
Pesanan: Mie Ayam sebanyak 3 porsi.
```

Pada contoh di atas, parameter `jumlah` memiliki nilai default `1`.

Parameter default, wajib diletakkan di urutan terakhir.

Contoh yang salah:

```
def pesan_makan(menu = 'Nasi Goreng', jumlah):  
    print(f"Pesanan: {menu} sebanyak {jumlah} porsi.")
```

Kode di atas akan menghasilkan `SyntaxError` karena parameter default (`menu`) diletakkan sebelum parameter yang tidak memiliki nilai default (`jumlah`).

Python mengharuskan semua parameter tanpa nilai default ditulis terlebih dahulu, kemudian diikuti oleh parameter yang memiliki nilai default.

Contoh yang benar:

```
def pesan_makan(menu, jumlah = 1):  
    print(f"Pesanan: {menu} sebanyak {jumlah} porsi.")
```

Aturan ini dibuat agar Python dapat menentukan dengan jelas nilai mana yang akan diberikan ke setiap parameter saat fungsi dipanggil.

Parameter Fleksibel

Pada umumnya, jumlah data yang diterima oleh fungsi harus sesuai dengan jumlah parameter yang telah ditentukan. Namun, dalam beberapa kasus kita mungkin tidak mengetahui berapa banyak data yang akan dikirim ke fungsi.

Untuk mengatasi hal tersebut, Python menyediakan parameter fleksibel, yaitu parameter yang dapat menerima jumlah data yang tidak terbatas.

Python menyediakan dua jenis parameter fleksibel, yaitu `*args` dan `**kwargs`.

Parameter `*args` (*Arguments*)

Parameter `*args` digunakan untuk menerima banyak data sekaligus tanpa menggunakan nama kunci (*keyword*).

Tanda bintang satu (*) di depan nama parameter memberi tahu Python bahwa semua data yang diterima akan dikumpulkan dan disimpan dalam bentuk `tuple`.

Contoh:

```
def cetak_hobi(*daftar_hobi):  
    print(daftar_hobi)  
    print(type(daftar_hobi))  
  
cetak_hobi("Membaca", "Main Game", "Menulis")
```

Output:

```
('Membaca', 'Main Game', 'Menulis')  
<class 'tuple'>
```

Dari output di atas, dapat diketahui bahwa seluruh data yang dikirim ke parameter `daftar_hobi` otomatis dibungkus menjadi sebuah [tuple](#).

Karena berupa [tuple](#), data tersebut dapat diakses menggunakan [perulangan](#).

Contoh:

```
def cetak_hobi(*daftar_hobi):  
    print("Hobi saya adalah:")  
    for hobi in daftar_hobi:  
        print(f"- {hobi}")  
  
cetak_hobi("Membaca", "Main Game", "Menulis")
```

Output:

```
Hobi saya adalah:  
- Membaca  
- Main Game  
- Menulis
```

Dengan `*args`, fungsi dapat menerima berapa pun jumlah data tanpa perlu menambahkan parameter baru.

Parameter `**kwargs` (*Keyword Arguments*)

Parameter `**kwargs` digunakan untuk menerima banyak data yang memiliki nama kunci (*keyword*).

Tanda bintang dua (`**`) di depan nama parameter memberi tahu Python bahwa semua data yang diterima akan dikumpulkan dan disimpan dalam bentuk [dictionary](#), yaitu pasangan kunci (*key*) dan nilai (*value*).

Contoh:

```
def tampilkan_profil(**data_diri):  
    print(data_diri)  
    print(type(data_diri))  
  
tampilkan_profil>Nama="Udin", Umur=20, Status="Mahasiswa")
```

Output:

```
{'Nama': 'Udin', 'Umur': 20, 'Status': 'Mahasiswa'}  
<class 'dict'>
```

Dari output di atas, dapat diketahui bahwa seluruh data yang dikirim ke parameter `data_diri` otomatis dibungkus menjadi sebuah [dictionary](#).

Karena berupa [dictionary](#), data dapat ditampilkan satu per satu menggunakan [perulangan](#).

Contoh:

```
def tampilkan_profil(**data_diri):  
    print("Profil Pengguna:")  
    for kunci, nilai in data_diri.items():  
        print(f"{kunci}: {nilai}")  
    print()  
  
tampilkan_profil>Nama="Udin", Umur=20, Status="Mahasiswa")  
tampilkan_profil>Nama="Siti", Kota="Bandung", Status="Mahasiswa")
```

Output:

```
Profil Pengguna:  
Nama: Udin  
Umur: 20  
Status: Mahasiswa  
  
Profil Pengguna:  
Nama: Siti  
Kota: Bandung  
Status: Mahasiswa
```

Dengan `**kwargs`, fungsi dapat menerima jumlah data yang tidak terbatas sekaligus tetap menyimpan nama setiap data yang dikirim.

Fungsi dengan Nilai Kembalian (*Return Function*)

Fungsi dengan pengembalian nilai (*return function*) adalah fungsi yang menghasilkan suatu nilai yang dapat digunakan kembali oleh program. Pengembalian nilai dilakukan menggunakan kata kunci `return`.

Ketika perintah `return` dijalankan, fungsi akan mengembalikan nilai yang ditentukan dan proses eksekusi fungsi akan berhenti.

Bentuk dasar:

```
def nama_fungsi(parameter):  
    return hasil
```

Contoh:

```
def tambah(a, b):  
    return a + b  
print(tambah(5, 3)) # Output: 8
```

Pada contoh di atas, fungsi `tambah()` mengembalikan hasil penjumlahan dari parameter `a` dan `b` menggunakan `return`.

Saat fungsi dipanggil dengan:

```
tambah(5, 3)
```

Parameter `a` akan berisi nilai `5` dan parameter `b` akan berisi nilai `3`. Kemudian fungsi mengembalikan hasil **penjumlahan** keduanya, yaitu `8`.

Nilai yang dikembalikan oleh fungsi tidak harus langsung ditampilkan menggunakan `print()`. Nilai tersebut juga dapat disimpan ke dalam variabel untuk digunakan kembali.

Contoh:

```
def tambah(a, b):  
    return a + b  
  
hasil = tambah(5, 3)  
print(hasil) # Output: 8
```

Dengan menggunakan `return`, hasil dari suatu fungsi dapat diproses kembali, disimpan ke dalam variabel, atau digunakan sebagai bagian dari operasi lainnya.

Scope Variabel

Scope variabel atau cakupan variabel adalah wilayah kode program tempat sebuah variabel dapat dikenali dan diakses.

Di Python, tidak semua variabel dapat diakses dari mana saja. Lokasi tempat variabel dibuat menentukan sejauh mana variabel tersebut dapat digunakan.

Secara umum, terdapat dua jenis *scope* variabel yang perlu diketahui, yaitu [variabel lokal](#) dan [variabel global](#).

Variabel Lokal

Variabel lokal adalah variabel yang dibuat di dalam sebuah fungsi dan hanya dapat diakses oleh perintah-perintah yang berada di dalam fungsi tersebut.

Contoh:

```
def cetak_angka():  
    # Variabel lokal  
    angka = 255  
    print(angka) # Bisa diakses di dalam fungsi  
  
cetak_angka()  
  
# print(angka)  
# ^ Jika dijalankan akan menghasilkan error,  
# karena variabel angka tidak dikenal di luar fungsi.
```

Output:

```
255
```

Pada contoh di atas, variabel `angka` dibuat di dalam fungsi `cetak_angka()`. Oleh karena itu, variabel tersebut hanya dapat digunakan di dalam fungsi tersebut.

Jika variabel `angka` diakses dari luar fungsi, Python akan menghasilkan `NameError` karena variabel tersebut berada di luar cakupannya (*scope*).

Variabel Global

Variabel global adalah variabel yang dibuat di luar fungsi dan dapat diakses dari mana saja, baik dari luar fungsi maupun dari dalam fungsi.

Contoh:

```
# Variabel global
warna = "Merah"

def cetak_warna():
    print(warna) # Bisa diakses dari dalam fungsi

cetak_warna()
print(warna) # Bisa diakses dari luar fungsi
```

Output:

```
Merah
Merah
```

Pada contoh di atas, variabel warna dibuat di luar fungsi sehingga menjadi variabel global. Karena bersifat global, variabel tersebut dapat diakses baik di dalam fungsi `cetak_warna()` maupun di luar fungsi.

Lambda (*Anonymous Function*)

Fungsi pada umumnya dibuat menggunakan kata kunci `def` dan memiliki nama. Namun, Python juga menyediakan fungsi lambda, yaitu fungsi yang tidak memiliki nama (*anonymous function*).

Fungsi lambda biasanya digunakan untuk membuat fungsi sederhana yang hanya terdiri dari satu ekspresi atau satu baris perhitungan.

Bentuk dasar:

```
nama_variabel = lambda parameter: proses_perhitungan
```

Hasil dari ekspresi yang ditulis setelah tanda titik dua (`:`) akan otomatis dikembalikan (*return*) oleh fungsi lambda. Oleh karena itu, fungsi lambda tidak memerlukan kata kunci `return`.

Contoh:

```
kuadrat = lambda x: x ** 2  
  
print(kuadrat(5))
```

Output:

```
25
```

Pada contoh di atas, fungsi lambda menerima satu parameter, yaitu `x`, kemudian mengembalikan hasil `x ** 2` atau kuadrat dari nilai tersebut.

Fungsi di atas setara dengan fungsi biasa berikut:

```
def kuadrat(x):  
    return x ** 2
```

Lambda dengan Banyak Parameter

Fungsi lambda juga dapat menerima lebih dari satu parameter.

Contoh:

```
kali = lambda a, b: a * b  
  
print(kali(4, 5))
```

Output:

```
20
```

Pada contoh di atas, fungsi lambda menerima dua parameter, yaitu `a` dan `b`, kemudian mengembalikan hasil [perkalian](#) keduanya.

Fungsi tersebut setara dengan:

```
def kali(a, b):  
    return a * b
```

Fungsi Bawaan (*Built-in Functions*)

Selain dapat membuat fungsi sendiri menggunakan `def`, Python juga menyediakan berbagai fungsi bawaan yang siap digunakan tanpa perlu membuatnya terlebih dahulu. Fungsi-fungsi tersebut disebut *Built-in Functions*.

Fungsi bawaan sangat membantu karena dapat digunakan untuk berbagai kebutuhan, seperti menerima input dari pengguna, menampilkan output, melakukan perhitungan, hingga mengolah kumpulan data.

Fungsi Input / Output (I/O)

Fungsi Input/Output (I/O) digunakan untuk berinteraksi dengan pengguna, baik untuk menampilkan informasi maupun menerima masukan.

`print()`

Fungsi `print()` digunakan untuk menampilkan data ke layar.

Contoh:

```
print("Halo Dunia")  
  
print("Teks pertama.", end=" ")  
print("Teks kedua.")
```

Output:

```
Halo Dunia  
Teks pertama. Teks kedua.
```

Pada contoh di atas, parameter `end=" "` digunakan untuk mencegah `print()` berpindah ke baris baru setelah menampilkan teks pertama.

`input()`

Fungsi `input()` digunakan untuk menerima masukan dari pengguna melalui keyboard.

Nilai yang dihasilkan oleh fungsi `input()` selalu bertipe data `string`, meskipun pengguna memasukkan angka.

Contoh:

```
nama = input("Masukkan nama Anda: ")
print(f"Halo, {nama}!")
print(type(nama))
```

Output:

```
Masukkan nama Anda: Ridwan
Halo, Ridwan!
<class 'str'>
```

Fungsi Iterasi dan Koleksi

Fungsi-fungsi pada kategori ini sering digunakan saat bekerja dengan perulangan (*looping*) atau kumpulan data seperti `string`, `list`, `tuple`, dan lainnya.

`range()`

Fungsi `range()` digunakan untuk menghasilkan deret angka dalam rentang tertentu.

Karena `range()` menghasilkan objek `range`, kita perlu mengubahnya menjadi `list` atau `tuple` jika ingin melihat seluruh isi deret angkanya.

Contoh:

```
print(range(5))
```

Output:

```
range(0, 5)
```

`range(stop)`

Menghasilkan angka dari 0 hingga `stop - 1`.

```
print(list(range(5)))
```

Output:

```
[0, 1, 2, 3, 4]
```

`range(start, stop)`

Menghasilkan angka dari `start` hingga `stop - 1`.

Contoh:

```
print(list(range(3, 7)))
```

Output:

```
[3, 4, 5, 6]
```

`range(start, stop, step)`

Menghasilkan angka dari `start` hingga `stop - 1` dengan langkah tertentu.

Contoh:

```
print(list(range(1, 10, 2)))
```

Output:

```
[1, 3, 5, 7, 9]
```

`len()`

Fungsi `len()` digunakan untuk menghitung jumlah elemen pada suatu objek. Pada [string](#), fungsi ini menghitung jumlah karakter.

Contoh:

```
nama = "Python"
huruf = ['a', 'b', 'c']

print(len(nama))
print(len(huruf))
```

Output:

```
6
3
```

enumerate()

Fungsi `enumerate()` digunakan untuk menghasilkan data beserta nomor urutnya dalam bentuk pasangan indeks dan nilai.

Contoh:

```
daftar_buah = ['apel', 'jeruk', 'mangga']

print(list(enumerate(daftar_buah)))
```

Output:

```
[(0, 'apel'), (1, 'jeruk'), (2, 'mangga')]
```

Fungsi ini sering digunakan pada [perulangan](#).

Contoh:

```
daftar_buah = ['apel', 'jeruk', 'mangga']

for indeks, buah in enumerate(daftar_buah):
    print(f"Indeks: {indeks}, Buah: {buah}")
```

Output:

```
Indeks: 0, Buah: apel
Indeks: 1, Buah: jeruk
Indeks: 2, Buah: mangga
```

Fungsi Matematika Dasar

Fungsi-fungsi berikut digunakan untuk melakukan perhitungan matematika tanpa harus menuliskan rumus secara manual.

abs()

Fungsi `abs(x)` digunakan untuk menghasilkan nilai mutlak (absolut) dari `x`.

Contoh:

```
x = -3
y = 9

print(abs(x))
print(abs(y))
```

Output:

```
3
9
```

pow()

Fungsi `pow(x, y)` digunakan untuk menghitung nilai pangkat `x` terhadap `y`.

Contoh:

```
x = 3
y = 2

print(pow(x, y))
```

Output:

```
9
```

Fungsi di atas setara dengan:

```
print(x ** y)
```

round()

Fungsi `round(x, n)` digunakan untuk membulatkan bilangan desimal hingga sejumlah digit tertentu.

Contoh:

```
print(round(3.14159, 2))
```

Output:

```
3.14
```

max()

Fungsi `max()` digunakan untuk mencari nilai terbesar dari beberapa nilai yang diberikan.

Contoh:

```
print(max(1, 4, 2, 8, 4))
```

Output:

```
8
```

min()

Fungsi `min()` digunakan untuk mencari nilai terkecil dari beberapa nilai yang diberikan.

Contoh:

```
print(min(1, 4, 2, 8, 4))
```

Output:

```
1
```

Bab 7: Pemrograman Berorientasi Objek

Pemrograman berorientasi objek atau *Object-Oriented Programming* (OOP) adalah metode pemrograman yang berfokus pada **objek**.

Dalam OOP, program dibangun menggunakan **objek** yang memiliki data dan perilaku. Pendekatan ini membuat kode lebih terstruktur, dan mudah dikembangkan ketika program menjadi lebih besar.

Class dan Objek

Dalam pemrograman berorientasi objek, terdapat dua konsep utama, yaitu **class** dan **objek**.

Class

Class adalah cetak biru (*blueprint*) yang digunakan untuk membuat **objek**.

Class mendefinisikan data dan perilaku yang akan dimiliki oleh **objek** yang dibuat darinya.

Contoh:

```
class Mobil:
    def __init__(self, merk, warna, tahun):
        self.merk = merk
        self.warna = warna
        self.tahun = tahun

    def info(self):
        return f"Mobil {self.merk} berwarna {self.warna} tahun {self.tahun}"
```

Pada contoh di atas, **Mobil** adalah nama class.

Class tersebut mendefinisikan bahwa setiap **objek** **Mobil** akan memiliki:

- **merk**
- **warna**
- **tahun**

serta sebuah **method** bernama **info()**.

Objek

Objek adalah hasil pembuatan dari sebuah `class`.

Jika `class` adalah cetak biru rumah, maka objek adalah rumah yang dibangun berdasarkan cetak biru tersebut.

Contoh:

```
mobil1 = Mobil("Toyota", "Merah", 2020)
```

Pada contoh di atas, `mobil1` adalah objek yang dibuat dari `class` `Mobil`.

Satu `class` dapat digunakan untuk membuat banyak objek.

Contoh:

```
mobil1 = Mobil("Toyota", "Merah", 2020)
mobil2 = Mobil("Honda", "Hitam", 2019)
```

Meskipun dibuat dari `class` yang sama, setiap objek dapat memiliki data yang berbeda.

Komponen Utama dalam Class

Sebuah `class` umumnya terdiri dari beberapa komponen penting, yaitu: `Konstruktor`, `Properti (atribut)`, dan `Method`.

Konstruktor

Konstruktor (*constructor*) adalah `method` khusus yang otomatis dijalankan saat `objek` dibuat.

Dalam Python, konstruktor ditulis menggunakan method `__init__()`.

Tugas utama konstruktor adalah menyiapkan data awal yang dimiliki `objek`.

Contoh:

```
class Mobil:
    def __init__(self, merk, warna, tahun):
        self.merk = merk
        self.warna = warna
        self.tahun = tahun
```

Pada contoh di atas, `__init__()` akan dijalankan secara otomatis ketika `objek Mobil` dibuat.

Misalnya:

```
mobil1 = Mobil("Toyota", "Merah", 2020)
```

Saat kode tersebut dijalankan, nilai `"Toyota"`, `"Merah"`, dan `2020` akan dimasukkan ke dalam `properti objek`.

Properti (Atribut)

Properti (*property*) atau atribut (*attribute*) adalah data yang dimiliki oleh `objek`.

Berdasarkan contoh sebelumnya:

```
class Mobil:
    def __init__(self, merk, warna, tahun):
        self.merk = merk
        self.warna = warna
        self.tahun = tahun
```

`Class Mobil` memiliki tiga properti:

- `merk`
- `warna`
- `tahun`

Nilai properti dapat diakses menggunakan tanda titik (`.`).

Setiap `objek` dapat memiliki nilai properti yang berbeda.

Contoh:

```
mobil1 = Mobil("Toyota", "Merah", 2020)
mobil2 = Mobil("Honda", "Hitam", 2019)

print(mobil1.merk)
print(mobil2.merk)
```

Output:

```
Toyota
Honda
```

Method

Method adalah [fungsi](#) yang berada di dalam [class](#).

Method digunakan untuk mendefinisikan perilaku atau aksi yang dapat dilakukan oleh [objek](#).

Contoh:

```
class Mobil:
    def __init__(self, merk, warna, tahun):
        self.merk = merk
        self.warna = warna
        self.tahun = tahun

    def info(self):
        return f"Mobil {self.merk} berwarna {self.warna} tahun {self.tahun}"
```

Pada contoh di atas, `info()` adalah sebuah method.

Method tersebut digunakan untuk menampilkan informasi mengenai [objek](#).

Contoh penggunaan:

```
mobil1 = Mobil("Toyota", "Merah", 2020)

print(mobil1.info())
```

Output:

```
Mobil Toyota berwarna Merah tahun 2020
```

Penggunaan `self`

`self` adalah `parameter` khusus yang digunakan untuk merujuk ke `objek` yang sedang digunakan.

Melalui `self`, sebuah `objek` dapat mengakses `properti` dan `method` miliknya sendiri.

Contoh:

```
class Mobil:
    def __init__(self, merk, warna, tahun):
        self.merk = merk
        self.warna = warna
        self.tahun = tahun

    def info(self):
        return f"Mobil {self.merk} berwarna {self.warna} tahun {self.tahun}"
```

Ketika `objek` dibuat:

```
mobil1 = Mobil("Toyota", "Merah", 2020)
mobil2 = Mobil("Honda", "Hitam", 2019)
```

Maka:

- `mobil1.merk` berisi "Toyota"
- `mobil2.merk` berisi "Honda"

Saat `method` dipanggil:

```
print(mobil1.info())
print(mobil2.info())
```

Output:

```
Mobil Toyota berwarna Merah tahun 2020
Mobil Honda berwarna Hitam tahun 2019
```

Pada pemanggilan `mobil1.info()`, `self` merujuk ke `objek` `mobil1`.

Sedangkan pada pemanggilan `mobil2.info()`, `self` merujuk ke `objek` `mobil2`.

Karena itulah `method` yang sama dapat menghasilkan output yang berbeda untuk setiap `objek`.

Membuat dan Mengubah Properti

Setelah **objek** dibuat, nilai **properti** bisa **diubah**, **ditambahkan**, ataupun **dihapus**.

Mengubah Nilai Properti

Nilai **properti** dapat diubah setelah **objek** dibuat.

Contoh:

```
class Mobil:
    def __init__(self, merk, warna, tahun):
        self.merk = merk
        self.warna = warna
        self.tahun = tahun

mobil1 = Mobil("Toyota", "Merah", 2020)
mobil1.warna = "Hitam"
print(mobil1.warna)
```

Output:

```
Hitam
```

Pada contoh di atas, nilai **properti** **warna** yang awalnya **"Merah"** diubah menjadi **"Hitam"**.

Menambahkan Properti Baru

Selain **properti** yang dibuat di dalam **konstruktor**, Python juga memungkinkan kita menambahkan **properti** baru langsung pada **objek**.

Contoh:

```
class Mobil:
    def __init__(self, merk, warna, tahun):
        self.merk = merk
        self.warna = warna
        self.tahun = tahun

mobil1 = Mobil("Toyota", "Merah", 2020)
mobil1.plat_nomor = "D 1234 ABC"
print(mobil1.plat_nomor)
```

Output:

```
D 1234 ABC
```

Pada contoh di atas, `properti plat_nomor` ditambahkan setelah `objek` dibuat.

Menghapus Properti

`Properti` juga dapat dihapus menggunakan perintah `del`.

Contoh:

```
class Mobil:
    def __init__(self, merk, warna, tahun):
        self.merk = merk
        self.warna = warna
        self.tahun = tahun

mobil1 = Mobil("Toyota", "Merah", 2020)
del mobil1.tahun
print(mobil1.tahun)
```

Output:

```
AttributeError: 'Mobil' object has no attribute 'tahun'
```

Error tersebut muncul karena `properti tahun` telah dihapus sehingga tidak lagi dimiliki oleh `objek`.

Class Attribute dan Instance Attribute

Dalam Python, terdapat dua jenis atribut yang umum digunakan, yaitu *class attribute* dan *instance attribute*.

Instance Attribute

Instance attribute adalah `atribut` atau `properti` yang dimiliki oleh setiap `objek`.

`Atribut` ini biasanya dibuat di dalam `konstruktor` (`__init__()`) menggunakan `self`.

Contoh:

```
class Mobil:
    def __init__(self, merk, warna):
        self.merk = merk
        self.warna = warna

mobil1 = Mobil("Toyota", "Merah")
mobil2 = Mobil("Honda", "Hitam")

print(mobil1.merk)
print(mobil2.merk)
```

Output:

```
Toyota
Honda
```

Pada contoh di atas, `merk` dan `warna` merupakan *instance attribute* karena didefinisikan menggunakan `self` di dalam *konstruktor* (`__init__()`).

Setiap *objek* memiliki nilai *atribut*nya masing-masing.

Class Attribute

Class attribute adalah *atribut* atau *properti* yang dimiliki oleh *class* dan digunakan bersama oleh seluruh *objek* yang dibuat dari *class* tersebut.

Class attribute didefinisikan langsung di dalam *class*, tetapi di luar *method*.

Contoh:

```
class Mobil:
    jumlah_roda = 4

    def __init__(self, merk):
        self.merk = merk

mobil1 = Mobil("Toyota")
mobil2 = Mobil("Honda")

print(mobil1.jumlah_roda)
print(mobil2.jumlah_roda)
```

Output:

```
4
4
```

Pada contoh di atas, `jumlah_roda` merupakan *class attribute* karena dibuat langsung di dalam *class*, bukan menggunakan *self* di dalam *konstruktor*.

Pilar-Pilar OOP

Pemrograman Berorientasi Objek memiliki empat konsep utama yang sering disebut sebagai empat pilar OOP, yaitu *Encapsulation* (Enkapsulasi), *Inheritance* (Pewarisan), *Polymorphism* (Polimorfisme), dan *Abstraction* (Abstraksi).

Encapsulation (Enkapsulasi)

Encapsulation (Enkapsulasi) adalah salah satu pilar OOP yang membungkus data (atribut) dan *method* (metode) ke dalam sebuah *class* serta melindungi data tersebut agar tidak diakses atau diubah secara sembarangan dari luar *class*.

Atribut Publik

Secara default, *atribut* yang dibuat menggunakan `self.nama_atribut` bersifat publik (*public*), sehingga dapat diakses langsung dari luar *class*.

Contoh:

```
class Mobil:
    def __init__(self, merk):
        self.merk = merk

mobil = Mobil("Toyota")
print(mobil.merk)
```

Output:

```
Toyota
```

Pada contoh di atas, atribut `merk` dapat diakses secara langsung karena merupakan *atribut publik*.

Atribut *Private*

Python menyediakan penulisan dengan dua garis bawah (__) di depan nama atribut untuk membuat atribut *private*.

Contoh:

```
class Mobil:
    def __init__(self, merk):
        self.__merk = merk

mobil = Mobil("Toyota")
print(mobil.__merk)
```

Output:

```
AttributeError: 'Mobil' object has no attribute '__merk'
```

Pada contoh di atas, atribut `__merk` tidak dapat diakses langsung dari luar class.

Mengakses Data *Private* dengan Method (Getter)

Karena atribut *private* tidak dapat diakses secara langsung, biasanya dibuat method untuk mengambil nilainya.

Method yang digunakan untuk mengambil nilai atribut disebut *getter*.

Contoh:

```
class Mobil:
    def __init__(self, merk):
        self.__merk = merk
    def get_merk(self):
        return self.__merk

mobil = Mobil("Toyota")
print(mobil.get_merk())
```

Output:

```
Toyota
```

Pada contoh di atas, method `get_merk()` digunakan untuk mengakses nilai atribut *private* `__merk`.

Mengubah Data *Private* dengan Method (Setter)

Selain mengambil data, kita juga dapat membuat **method** untuk mengubah nilai **atribut *private***.

Method yang digunakan untuk mengubah nilai **atribut** disebut *setter*.

Contoh:

```
class Mobil:
    def __init__(self, merk):
        self.__merk = merk

    def get_merk(self):
        return self.__merk
    def set_merk(self, merk):
        self.__merk = merk

mobil = Mobil("Toyota")
mobil.set_merk("Honda")
print(mobil.get_merk())
```

Output:

```
Honda
```

Pada contoh di atas, **method** `set_merk()` digunakan untuk mengubah nilai **atribut `__merk`** dari "Toyota" menjadi "Honda".

Inheritance (Pewarisan)

Inheritance (Pewarisan) adalah salah satu pilar OOP yang memungkinkan sebuah **class** mewarisi atribut dan method dari **class** lain.

Class yang memberikan warisan disebut *parent class* (class induk), sedangkan **class** yang menerima warisan disebut *child class* (class turunan).

Dengan *inheritance*, kita tidak perlu menulis ulang **atribut** dan **method** yang sudah ada pada class induk sehingga kode menjadi lebih ringkas dan mudah dikelola.

Membuat Child Class

Child class atau kelas turunan (subclass) adalah **class** yang mewarisi **atribut** dan **method** dari *parent class* atau kelas induk.

Contoh:

```
class Kendaraan:
    def bergerak(self):
        print("Kendaraan sedang bergerak")

class Mobil(Kendaraan):
    pass

mobil = Mobil()
mobil.bergerak()
```

Output:

```
Kendaraan sedang bergerak
```

Pada contoh di atas, `class Mobil` mewarisi `class Kendaraan`.

Meskipun `class Mobil` tidak memiliki `method bergerak()`, objek `mobil` tetap dapat menggunakannya karena `method` tersebut diwarisi dari `class Kendaraan`.

Mewarisi Atribut dan Method

Inheritance tidak hanya mewarisi `method`, tetapi juga `atribut`.

Contoh:

```
class Kendaraan:
    def __init__(self, merk):
        self.merk = merk
    def info(self):
        print(f"Merk kendaraan: {self.merk}")

class Mobil(Kendaraan):
    pass

mobil = Mobil("Toyota")
mobil.info()
```

Output:

```
Merk kendaraan: Toyota
```

Pada contoh di atas, `class Mobil` mewarisi `atribut merk` dan `method info()` dari `class Kendaraan`.

Menambahkan Method Baru pada Child Class

Class turunan dapat memiliki **method** tambahan yang tidak dimiliki oleh class induk.

Contoh:

```
class Kendaraan:
    def bergerak(self):
        print("Kendaraan sedang bergerak")

class Mobil(Kendaraan):
    def maju(self):
        print("Mobil sedang maju")

mobil = Mobil()

mobil.bergerak()
mobil.maju()
```

Output:

```
Kendaraan sedang bergerak
Mobil sedang maju
```

Pada contoh di atas, **class** `Mobil` mewarisi **method** `bergerak()` dari **class** `Kendaraan` dan memiliki **method** tambahan yaitu `maju()`.

Penggunaan `super()`

Fungsi `super()` digunakan untuk mengakses **atribut** atau **method** yang dimiliki oleh *parent class* (class induk).

Salah satu penggunaan `super()` yang paling umum adalah untuk memanggil **konstruktor** (`__init__()`) dari class induk.

Contoh:

```
class Kendaraan:
    def __init__(self, merk):
        self.merk = merk

class Mobil(Kendaraan):
    def __init__(self, merk, warna):
        super().__init__(merk)
        self.warna = warna

mobil = Mobil("Toyota", "Merah")
print(mobil.merk)
print(mobil.warna)
```

Output:

```
Toyota
Merah
```

Pada contoh di atas, `super().__init__(merk)` digunakan untuk memanggil [konstruktor](#) milik [class](#) `Kendaraan`.

Dengan demikian, atribut `merk` dapat dibuat oleh class induk, sedangkan [atribut](#) `warna` dibuat oleh class `Mobil`.

Tanpa `super()`, kita harus menuliskan kembali kode yang ada pada [konstruktor](#) class induk.

Contoh:

```
class Kendaraan:
    def __init__(self, merk):
        self.merk = merk

class Mobil(Kendaraan):
    def __init__(self, merk, warna):
        self.merk = merk
        self.warna = warna
```

Meskipun menghasilkan output yang sama, cara tersebut kurang disarankan karena menimbulkan duplikasi kode.

Dengan menggunakan `super()`, kode menjadi lebih ringkas dan lebih mudah dipelihara.

Polymorphism (Polimorfisme)

Polymorphism (Polimorfisme) adalah salah satu pilar OOP yang memungkinkan [method](#) yang sama memiliki perilaku yang berbeda pada [objek](#) yang berbeda.

Kata *polymorphism* berasal dari bahasa Yunani yang berarti "memiliki banyak bentuk".

Dengan *polymorphism*, kita dapat menggunakan nama [method](#) yang sama pada beberapa [class](#), tetapi masing-masing [class](#) dapat memberikan implementasi yang berbeda.

Contoh:

```
class Kendaraan:
    def bergerak(self):
        print("Kendaraan sedang bergerak")

class Mobil(Kendaraan):
    def bergerak(self):
        print("Mobil berjalan di jalan raya")

class Pesawat(Kendaraan):
    def bergerak(self):
        print("Pesawat terbang di udara")

mobil = Mobil()
pesawat = Pesawat()
mobil.bergerak()
pesawat.bergerak()
```

Output:

```
Mobil berjalan di jalan raya
Pesawat terbang di udara
```

Pada contoh di atas, [class](#) `Mobil` dan `Pesawat` memiliki [method](#) `bergerak()` yang sama.

Namun, ketika [method](#) tersebut dipanggil, masing-masing [objek](#) memberikan perilaku yang berbeda.

Abstraction (Abstraksi)

Abstraction (Abstraksi) adalah salah satu pilar OOP yang berfungsi menyembunyikan detail implementasi dan hanya menampilkan fungsi-fungsi penting kepada pengguna.

Dengan abstraksi, pengguna cukup mengetahui apa yang dapat dilakukan oleh suatu objek tanpa perlu mengetahui bagaimana proses tersebut dilakukan di dalam class.

Penggunaan Abstract Class

Python menyediakan modul `abc` (*Abstract Base Class*) untuk membuat *abstract class*.

Abstract class adalah `class` yang tidak digunakan untuk membuat objek secara langsung, melainkan sebagai dasar bagi `class` lain. Biasanya, *abstract class* berperan sebagai *parent class* yang mendefinisikan method-method yang wajib diimplementasikan oleh class turunannya.

Contoh:

```
from abc import ABC, abstractmethod

class Kendaraan(ABC):
    @abstractmethod
    def bergerak(self):
        pass

class Mobil(Kendaraan):
    def bergerak(self):
        print("Mobil berjalan di jalan raya")

mobil = Mobil()
mobil.bergerak()
```

Output:

```
Mobil berjalan di jalan raya
```

Pada Contoh di atas:

- `Kendaraan` adalah *abstract class*.
- `bergerak()` adalah *abstract method*.

- Class `Mobil` wajib membuat implementasi `method` `bergerak()`.

Abstract class tidak dapat langsung dijadikan `objek`.

Contoh:

```
from abc import ABC, abstractmethod

class Kendaraan(ABC):
    @abstractmethod
    def bergerak(self):
        pass

kendaraan = Kendaraan()
```

Output:

```
TypeError: Can't instantiate abstract class Kendaraan without an implementation for abstract method 'bergerak'
```

Error tersebut terjadi karena `Kendaraan` merupakan *abstract class* yang masih memiliki *abstract method* `bergerak()` yang belum diimplementasikan.

Oleh karena itu, `objek` harus dibuat dari class turunan yang telah menyediakan implementasi `method` tersebut, seperti `class Mobil`.

Bab 8: Penanganan Error

Error adalah kesalahan, bug, atau kondisi tidak terduga yang menyebabkan program gagal atau berjalan tidak sebagaimana mestinya.

Penanganan error (*error handling*) bertujuan untuk mengantisipasi, mendeteksi, dan merespons kesalahan yang terjadi saat program berjalan sehingga program tidak berhenti secara mendadak (*crash*).

Contoh tanpa penanganan error:

```
angka1 = 7
angka2 = 0

hasil = angka1 / angka2

print("Hasil pembagian:", hasil)
print("Program selesai.")
```

Output:

```
ZeroDivisionError: division by zero
```

Program di atas menghasilkan error karena melakukan **pembagian** dengan nol (0). Saat Python menemukan kesalahan pada baris:

```
hasil = angka1 / angka2
```

eksekusi program langsung dihentikan sehingga baris-baris kode setelahnya tidak dijalankan.

Oleh karena itu, diperlukan penanganan error agar program tetap dapat berjalan meskipun terjadi kesalahan.

Jenis-Jenis Error

Sebelum mempelajari cara menangani error, kita perlu memahami terlebih dahulu jenis-jenis error yang dapat terjadi dalam Python.

Syntax Error (Kesalahan Penulisan)

Error jenis ini akan terdeteksi sebelum program dijalankan. Selama masih terdapat *syntax error*, program tidak dapat dieksekusi sama sekali.

Contoh:

```
angka pertama = 2
```

Output:

```
SyntaxError: invalid syntax (error_handling.py, line 1)
```

Error terjadi karena nama variabel tidak boleh mengandung spasi.

Contoh lainnya:

```
print("Halo"
```

Output:

```
SyntaxError: '(' was never closed (error_handling.py, line 1)
```

Error terjadi karena tanda kurung penutup `)` tidak dituliskan.

Runtime Error (Kesalahan Saat Program Berjalan)

Runtime Error adalah error yang terjadi ketika program sedang dijalankan.

Pada kasus ini, penulisan kode sudah benar sehingga program dapat dijalankan. Namun, saat eksekusi berlangsung, Python menemukan kondisi yang menyebabkan program tidak dapat melanjutkan prosesnya.

Jenis-Jenis *Runtime Error* yang Sering Terjadi

Dalam Python, setiap *runtime error* memiliki nama atau tipe yang spesifik tergantung dari penyebab kesalahannya.

ZeroDivisionError

Error ini terjadi karena angka tidak dapat dibagi dengan nol.

Contoh:

```
hasil = 2 / 0
```

Output:

```
ZeroDivisionError: division by zero
```

ValueError

Error ini terjadi ketika suatu fungsi menerima data dengan tipe yang benar, tetapi nilai (value) di dalamnya tidak valid untuk diproses.

Contoh:

```
angka = int("dua puluh")
```

Output:

```
ValueError: invalid literal for int() with base 10: 'dua puluh'
```

Error dari contoh di atas terjadi karena fungsi `int()` hanya dapat mengubah teks yang berisi angka menjadi bilangan bulat.

TypeError

Error ini terjadi ketika kita melakukan operasi matematika atau fungsi pada dua tipe data yang tidak cocok.

```
hasil = "Lima" + 5 # Error karena teks tidak bisa dijumlahkan dengan angka
```

Output:

```
TypeError: can only concatenate str (not "int") to str
```

IndexError

Error ini terjadi ketika kita mencoba mengambil data dari sebuah [daftar \(list\)](#) menggunakan [nomor urut \(indeks\)](#) yang tidak ada atau di luar jangkauan.

Contoh:

```
buah = ["Apel", "Jeruk"]  
print(buah[5]) # Error karena indeks ke-5 tidak ada (list hanya sampai indeks 1)
```

Output:

```
IndexError: list index out of range
```

Logical Error (Kesalahan Logika)

Logical Error adalah kesalahan yang terjadi pada logika program.

Program tetap berjalan tanpa menampilkan pesan error, tetapi hasil yang diperoleh tidak sesuai dengan yang diharapkan.

Kesalahan ini biasanya terjadi karena rumus, kondisi, atau alur program yang dibuat tidak tepat.

Contoh:

```
nilai = 85

if nilai < 70:
    print("Selamat, Anda Lulus!")
else:
    print("Maaf, Anda Tidak Lulus.")
```

Output:

```
Maaf, Anda Tidak Lulus.
```

Pada contoh tersebut, program menghasilkan output yang salah. Dengan nilai 85, seharusnya siswa dinyatakan lulus.

Kesalahan terjadi karena kondisi yang digunakan tidak tepat. **Operator** yang benar seharusnya **lebih dari sama dengan (>=)**, Karena logika program salah, hasil yang ditampilkan juga menjadi tidak sesuai meskipun program berjalan tanpa error.

Penanganan Error dengan `try` dan `except`

Python menyediakan blok `try` dan `except` untuk menangani error yang mungkin terjadi saat program dijalankan.

Perlu diperhatikan bahwa `try` dan `except` hanya dapat digunakan untuk menangani **Runtime Error**, yaitu error yang terjadi saat program sedang dieksekusi. Error jenis ini muncul ketika Python menemukan kondisi yang tidak dapat diproses, seperti pembagian dengan nol, konversi tipe data yang tidak valid, atau akses ke indeks yang tidak tersedia.

Sementara itu, *Syntax Error* tidak dapat ditangani menggunakan `try` dan `except` karena program gagal diperiksa dan dijalankan sejak awal akibat kesalahan penulisan kode. Adapun *Logical Error* juga tidak dapat ditangani dengan `try` dan `except` karena program sebenarnya berjalan tanpa menghasilkan error, hanya saja logika yang dibuat menghasilkan output yang tidak sesuai dengan yang diharapkan. Oleh karena itu, mekanisme `try` dan `except` secara khusus digunakan untuk menangani *Runtime Error*.

Contoh:

```
try:
    angka1 = 7
    angka2 = 0

    hasil = angka1 / angka2
    print("Hasil pembagian:", hasil)
except:
    print("Terjadi kesalahan.")

print("Program selesai.")
```

Output:

```
Terjadi kesalahan.
Program selesai.
```

Pada contoh di atas, kode yang berpotensi menghasilkan error ditempatkan di dalam blok `try`.

Jika terjadi error, Python tidak langsung menghentikan program. Sebaliknya, eksekusi akan dialihkan ke blok `except` untuk menjalankan perintah penanganan yang telah disediakan.

Setelah blok `except` selesai dieksekusi, program akan melanjutkan eksekusi ke baris kode berikutnya hingga program selesai dijalankan.

Penanganan Error Tertentu pada `except`

Pada contoh sebelumnya, blok `except` digunakan tanpa menyebutkan jenis error yang ingin ditangani.

Contoh:

```
try:
    angka = 10 / 0
except:
    print("Terjadi kesalahan.")
```

Cara tersebut memang dapat menangani semua *Runtime Error*, tetapi tidak disarankan karena kita tidak mengetahui jenis error yang sebenarnya terjadi.

Python memungkinkan kita menangani jenis error tertentu dengan menuliskan nama error setelah kata kunci `except`.

Bentuk dasar:

```
try:
    # kode yang berpotensi menghasilkan error
except NamaError:
    # penanganan jika terjadi NamaError
```

Sebagai contoh, untuk menangani error akibat pembagian dengan nol, kita dapat menggunakan `ZeroDivisionError`.

```
try:
    angka1 = 10
    angka2 = 0

    hasil = angka1 / angka2
    print(hasil)
except ZeroDivisionError:
    print("Tidak dapat melakukan pembagian dengan nol.")
```

Output:

```
Tidak dapat melakukan pembagian dengan nol.
```

Pada contoh di atas, saat terjadi `ZeroDivisionError`, Python langsung menjalankan kode di dalam blok `except ZeroDivisionError`. Program tidak

berhenti secara tiba-tiba sehingga dapat terus melanjutkan eksekusi ke baris berikutnya.

Penggunaan `else` pada Penanganan Error

Selain menggunakan `except`, Python juga menyediakan blok `else` yang dapat digunakan bersama `try` dan `except`.

Blok `else` akan dieksekusi hanya jika tidak terjadi error di dalam blok `try`. Dengan kata lain, apabila seluruh kode pada `try` berhasil dijalankan tanpa menghasilkan error, Python akan melewati blok `except` dan langsung menjalankan blok `else`.

Bentuk dasar:

```
try:
    # kode yang berpotensi menghasilkan error
except NamaError:
    # dijalankan jika terjadi error
else:
    # dijalankan jika tidak terjadi error
```

Contoh:

```
try:
    angka1 = 10
    angka2 = 2

    hasil = angka1 / angka2
except ZeroDivisionError:
    print("Tidak dapat melakukan pembagian dengan nol.")
else:
    print("Hasil pembagian:", hasil)
```

Output:

```
Hasil pembagian: 5.0
```

Pada contoh di atas, tidak terjadi error karena `10` dapat dibagi dengan `2`. Oleh karena itu, blok `except` dilewati dan blok `else` dijalankan untuk menampilkan hasil pembagian.

Sebaliknya, jika nilai `angka2` diubah menjadi `0`, maka akan terjadi `ZeroDivisionError`. Dalam kondisi tersebut, blok `except` akan dijalankan, sedangkan blok `else` tidak akan dieksekusi.

Contoh:

```
try:
    angka1 = 10
    angka2 = 0

    hasil = angka1 / angka2
except ZeroDivisionError:
    print("Tidak dapat melakukan pembagian dengan nol.")
else:
    print("Hasil pembagian:", hasil)
```

Output:

```
Tidak dapat melakukan pembagian dengan nol.
```

Blok `else` sangat berguna untuk menempatkan kode yang hanya boleh dijalankan ketika proses pada blok `try` berhasil tanpa error. Dengan demikian, kode menjadi lebih rapi karena penanganan error dipisahkan dari kode yang dijalankan saat proses berhasil.

Penggunaan `finally` pada Penanganan Error

Python juga menyediakan blok `finally` yang dapat digunakan bersama `try` dan `except`.

Blok `finally` akan selalu dieksekusi, baik terjadi error maupun tidak terjadi error di dalam blok `try`. Oleh karena itu, blok ini biasanya digunakan untuk menjalankan proses pembersihan (*cleanup*), seperti menutup file, menutup koneksi database, atau melepaskan sumber daya yang sedang digunakan.

Bentuk dasar:

```
try:
    # kode yang berpotensi menghasilkan error
except NamaError:
    # dijalankan jika terjadi error
finally:
    # selalu dijalankan
```

Contoh saat tidak terjadi error:

```
try:
    angka1 = 10
    angka2 = 2

    hasil = angka1 / angka2
    print("Hasil pembagian:", hasil)
except ZeroDivisionError:
    print("Tidak dapat melakukan pembagian dengan nol.")
finally:
    print("Program selesai.")
```

Output:

```
Hasil pembagian: 5.0
Program selesai.
```

Pada contoh di atas, tidak terjadi error sehingga blok `except` dilewati. Setelah seluruh kode pada blok `try` selesai dijalankan, Python tetap mengeksekusi blok `finally`.

Contoh saat terjadi error:

```
try:
    angka1 = 10
    angka2 = 0

    hasil = angka1 / angka2
    print("Hasil pembagian:", hasil)
except ZeroDivisionError:
    print("Tidak dapat melakukan pembagian dengan nol.")
finally:
    print("Program selesai.")
```

Output:

```
Tidak dapat melakukan pembagian dengan nol.  
Program selesai.
```

Pada contoh ini terjadi `ZeroDivisionError` sehingga blok `except` dijalankan. Setelah itu, Python tetap mengeksekusi blok `finally` sebelum program benar-benar selesai.

Penggunaan `raise`

Perintah `raise` digunakan untuk memicu (melempar) error secara sengaja.

Berbeda dengan error yang muncul secara otomatis dari Python, `raise` memungkinkan programmer membuat atau memunculkan error berdasarkan kondisi yang ditentukan sendiri.

`raise` dapat digunakan di dalam blok `try` maupun di dalam blok `except`, tetapi tujuan penggunaannya berbeda.

Penggunaan `raise` di Dalam `try`

Ketika digunakan di dalam blok `try`, `raise` berfungsi untuk memicu error secara sengaja berdasarkan kondisi tertentu yang kita buat sendiri.

Teknik ini biasanya digunakan untuk melakukan validasi sebelum Python menghasilkan error secara otomatis. Dengan demikian, kita dapat memberikan pesan error yang lebih jelas dan sesuai dengan kebutuhan program.

Contoh:

```
try:  
    angka1 = 10  
    angka2 = 0  
  
    if angka2 == 0:  
        raise ValueError("Angka pembagi tidak boleh nol!")  
  
    hasil = angka1 / angka2  
    print("Hasil:", hasil)  
except ValueError as e:  
    print(f"Validasi gagal: {e}")
```

Output:

```
Validasi gagal: Angka pembagi tidak boleh nol!
```

Pada contoh di atas, kondisi `angka2 == 0` diperiksa terlebih dahulu. Jika kondisi tersebut bernilai `True`, maka `raise` akan memunculkan `ValueError` dengan pesan `"Angka pembagi tidak boleh nol!"`.

Error tersebut kemudian ditangkap oleh blok `except ValueError` sehingga program tidak berhenti secara tiba-tiba.

Penggunaan `raise` di Dalam `except`

Selain digunakan untuk membuat error baru, `raise` juga dapat digunakan di dalam blok `except`.

Dalam kondisi ini, `raise` berfungsi untuk melempar kembali (*re-raise*) error yang telah ditangkap, sehingga error tersebut dapat diteruskan ke pemanggil (*caller*) atau ditangani oleh blok `except` di tingkat yang lebih tinggi.

Contoh tanpa `raise`:

```
def cetak_pembagian(a, b):
    try:
        hasil = a / b
        print(f"Hasil pembagian {a} / {b} = {hasil}")
    except ZeroDivisionError:
        print("Error: Pembagian dengan nol tidak diperbolehkan.")

try:
    cetak_pembagian(10, 0)
    cetak_pembagian(10, 2)
except ZeroDivisionError:
    print("Terjadi kesalahan saat melakukan pembagian.")
```

Output:

```
Error: Pembagian dengan nol tidak diperbolehkan.
Hasil pembagian 10 / 2 = 5.0
```

Pada contoh di atas, `ZeroDivisionError` sudah ditangani di dalam `fungsi cetak_pembagian()`. Karena error tersebut tidak diteruskan kembali, program menganggap masalah telah selesai sehingga tetap melanjutkan pemanggilan:

```
cetak_pembagian(10, 2)
```

Akibatnya, blok `except` di luar `fungsi` tidak pernah dijalankan.

Contoh dengan `raise`:

```
def cetak_pembagian(a, b):
    try:
        hasil = a / b
        print(f"Hasil pembagian {a} / {b} = {hasil}")
    except ZeroDivisionError:
        print("Error: Pembagian dengan nol tidak diperbolehkan.")
        raise

try:
    cetak_pembagian(10, 0)
    cetak_pembagian(10, 2)
except ZeroDivisionError:
    print("Terjadi kesalahan saat melakukan pembagian.")
```

Output:

```
Error: Pembagian dengan nol tidak diperbolehkan.
Terjadi kesalahan saat melakukan pembagian.
```

Pada contoh di atas, setelah mencetak pesan error, perintah `raise` melempar kembali `ZeroDivisionError` yang telah ditangkap.

Karena error diteruskan ke blok `except` di luar `fungsi`, eksekusi program langsung berpindah ke:

```
except ZeroDivisionError:
    print("Terjadi kesalahan saat melakukan pembagian.")
```

Akibatnya, baris berikutnya:

```
cetak_pembagian(10, 2)
```

tidak dijalankan, karena proses eksekusi langsung dihentikan dan dialihkan ke blok `except` di luar `fungsi`.

Membuat Exception Kustom

Selain menggunakan exception¹² bawaan Python, kita juga dapat membuat exception kustom (*custom exception*) sesuai dengan kebutuhan program.

Exception kustom dibuat dengan cara membuat sebuah `class` baru yang merupakan turunan (*inheritance*) dari class `Exception`.

Cara ini sangat berguna ketika kita ingin memberikan nama error yang lebih spesifik sehingga penyebab kesalahan menjadi lebih mudah dipahami.

bentuk dasar:

```
class NamaException(Exception):  
    pass
```

`Class` tersebut dapat digunakan seperti exception bawaan Python, yaitu dengan perintah `raise` untuk memunculkan error dan `except` untuk menanganinya.

Contoh:

```
class ErrorNilaiNegatif(Exception):  
    def __init__(self):  
        message = "Terjadi kesalahan: Nilai negatif tidak diperbolehkan."  
        super().__init__(message)  
  
def cetak_akar_kuadrat(angka):  
    if angka < 0:  
        raise ErrorNilaiNegatif()  
    return angka ** 0.5  
  
try:  
    hasil = cetak_akar_kuadrat(-4)  
    print(f"Akar kuadrat: {hasil}")  
except ErrorNilaiNegatif as error:  
    print(error)
```

Output:

```
Terjadi kesalahan: Nilai negatif tidak diperbolehkan.
```

¹² Exception adalah objek yang merepresentasikan sebuah error atau kondisi tidak normal yang terjadi saat program dijalankan.

Pada contoh di atas, dibuat sebuah exception kustom bernama `ErrorNilaiNegatif` yang merupakan turunan dari `class Exception`.

```
class ErrorNilaiNegatif(Exception):
```

Di dalam `class` tersebut terdapat `method __init__()` yang digunakan untuk memberikan pesan error ketika exception dibuat.

```
def __init__(self):
    message = "Terjadi kesalahan: Nilai negatif tidak diperbolehkan."
    super().__init__(message)
```

Selanjutnya, pada fungsi `cetak_akar_kuadrat()`, dilakukan pemeriksaan terhadap nilai `parameter` angka.

```
if angka < 0:
    raise ErrorNilaiNegatif()
```

Jika nilai angka kurang dari 0, maka `raise` akan memunculkan exception `ErrorNilaiNegatif`.

Exception tersebut kemudian ditangkap oleh blok:

```
except ErrorNilaiNegatif as error:
    print(error)
```

Sehingga pesan error yang telah dibuat di dalam `class` akan ditampilkan ke layar.

Bab 9: Modul dan Package

Modul

Modul adalah sebuah file Python (dengan ekstensi `.py`) yang berisi kode, seperti `variabel`, `fungsi`, atau `class`, yang dapat digunakan kembali pada file Python lainnya.

Dengan menggunakan modul, kita dapat memisahkan program menjadi beberapa file sehingga kode menjadi lebih rapi, mudah dikelola, dan dapat digunakan kembali.

Membuat dan Mengimpor Modul

Membuat Modul

Di Python, kita dapat membuat modul sendiri. Caranya cukup dengan membuat sebuah file Python (`.py`) yang berisi `variabel`, `fungsi`, atau `class`.

Misalnya, kita membuat sebuah file bernama `matematika.py`.

```
PI = 3.14

def tambah(a, b):
    return a + b

class Kalkulator:
    pass
```

Modul tersebut kemudian dapat digunakan pada file Python lainnya dengan cara diimpor (*import*).

Mengimpor Modul

Python menyediakan beberapa cara untuk mengimpor modul, tergantung pada kebutuhan.

Mengimpor Seluruh Modul

Cara paling dasar untuk mengimpor modul adalah menggunakan perintah `import`.

Dengan cara ini, seluruh isi modul akan tersedia, tetapi setiap `variabel`, `fungsi`, atau `class` harus diakses melalui nama modulnya.

```
import matematika

print(matematika.tambah(5, 3)) # Output: 8
print(matematika.PI) # Output: 3.14

kalkulator = matematika.Kalkulator()
```

Pada contoh di atas:

- `matematika.tambah()` digunakan untuk memanggil fungsi `tambah()`.
- `matematika.PI` digunakan untuk mengakses variabel `PI`.
- `matematika.Kalkulator()` digunakan untuk membuat objek dari `class Kalkulator`.

Mengimpor Modul dengan Alias

Kita juga dapat memberikan nama lain (*alias*) pada modul menggunakan kata kunci `as`.

Cara ini berguna jika nama modul terlalu panjang atau ingin dibuat lebih singkat.

```
import matematika as mtk

print(mtk.tambah(5, 3)) # Output: 8
print(mtk.PI) # Output: 3.14

kalkulator = mtk.Kalkulator()
```

Pada contoh di atas, modul `matematika` diubah menjadi `mtk`, sehingga seluruh isi modul diakses menggunakan nama alias tersebut.

Mengimpor Isi Modul Secara Spesifik

Jika hanya membutuhkan beberapa bagian dari sebuah modul, kita dapat menggunakan `from ... import ...`

Dengan cara ini, hanya `variabel`, `fungsi`, atau `class` yang dipilih saja yang akan diimpor sehingga tidak perlu menuliskan nama modul saat menggunakannya.

```
from matematika import tambah, PI

print(tambah(5, 3)) # Output: 8
print(PI) # Output: 3.14
```

Selain **fungsi** dan **variabel**, kita juga dapat mengimpor **class**.

```
from matematika import Kalkulator

kalkulator = Kalkulator()
```

Kita juga dapat mengimpor seluruh isi modul sekaligus menggunakan tanda bintang (*).

```
from matematika import *

print(tambah(5, 3)) # Output: 8

print(PI) # Output: 3.14
```

Meskipun cara ini praktis, penggunaannya tidak disarankan karena dapat menyebabkan benturan nama (*name conflict*) jika terdapat **variabel** atau **fungsi** dengan nama yang sama. Selain itu, kode juga menjadi lebih sulit dibaca karena tidak terlihat dengan jelas asal **variabel**, **fungsi**, atau **class** yang digunakan. Oleh karena itu, sebaiknya gunakan `import modul` atau `from modul import nama` sesuai kebutuhan.

Modul Bawaan (*Built-in Modules*)

Selain membuat modul sendiri, Python juga menyediakan banyak modul bawaan (*built-in modules*) yang dapat langsung digunakan tanpa perlu menginstalnya terlebih dahulu. Modul-modul ini telah tersedia sejak Python pertama kali diinstal.

Berikut beberapa modul bawaan yang paling sering digunakan.

Modul `math`

Modul `math` digunakan untuk melakukan operasi matematika yang lebih kompleks, seperti menghitung akar kuadrat, fungsi trigonometri, logaritma, dan lain sebagainya.

```
import math

print(math.pi) # 3.141592653589793
print(math.sqrt(64)) # 8.0
print(math.ceil(4.2)) # 5
```

Modul datetime

Modul `datetime` digunakan untuk bekerja dengan tanggal dan waktu.

```
from datetime import datetime

sekarang = datetime.now()

print(sekarang) # Menampilkan tanggal dan waktu saat ini
print(sekarang.year) # Menampilkan tahun
```

Modul random

Modul `random` digunakan untuk menghasilkan angka acak atau memilih data secara acak.

```
import random

print(random.randint(1, 10)) # Angka acak antara 1 sampai 10

buah = ["apel", "pisang", "jeruk"]
print(random.choice(buah)) # Memilih satu buah secara acak
```

Modul os

Modul `os` digunakan untuk berinteraksi dengan sistem operasi, misalnya membuat folder, melihat direktori kerja, atau mengelola file dan folder.

```
import os

print(os.getcwd()) # Menampilkan direktori kerja saat ini

os.mkdir("folder_baru") # Membuat folder baru
```

Package

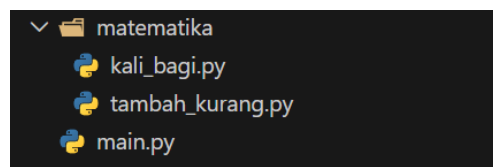
Package adalah sebuah folder yang berisi kumpulan modul Python. Package digunakan untuk mengelompokkan **modul-modul** yang saling berkaitan sehingga struktur program menjadi lebih rapi, mudah dikelola, dan lebih mudah dikembangkan.

Membuat dan Mengimpor Package

Membuat Package

Untuk membuat sebuah package, kita cukup membuat sebuah folder yang berisi satu atau lebih modul Python (`.py`).

Sebagai contoh, kita akan membuat package bernama `matematika` dengan struktur berikut.



Isi file `tambah_kurang.py`.

```
def tambah(a, b):  
    return a + b  
  
def kurang(a, b):  
    return a - b
```

Isi file `kali_bagi.py`.

```
def kali(a, b):  
    return a * b  
  
def bagi(a, b):  
    return a / b
```

Package tersebut kemudian dapat digunakan pada file Python lain dengan cara diimpor.

Mengimpor Package

Python menyediakan beberapa cara untuk mengimpor isi dari sebuah package.

Mengimpor Modul

Jika ingin menggunakan seluruh isi dari sebuah modul di dalam package, kita dapat menggunakan format berikut.

```
from nama_package import nama_modul
```

Contoh:

```
# file: main.py

from matematika import tambah_kurang

print(tambah_kurang.tambah(5, 3)) # Output: 8
print(tambah_kurang.kurang(5, 3)) # Output: 2
```

Pada contoh di atas, fungsi dipanggil melalui nama modul, yaitu `tambah_kurang`.

Mengimpor Bagian Modul Secara Spesifik

Jika hanya membutuhkan `fungsi`, `variabel`, atau `class` tertentu dari sebuah `modul` di dalam package, kita dapat menggunakan format berikut.

```
from nama_package.nama_modul import nama
```

Contoh:

```
# file: main.py

from matematika.kali_bagi import kali

print(kali(4, 2)) # Output: 8
```

Karena fungsi `kali()` diimpor secara langsung, kita dapat memanggilnya tanpa menuliskan nama modul terlebih dahulu.

PyPI

Selain membuat package sendiri, Python juga memiliki ribuan package yang dibuat oleh komunitas dan dapat digunakan secara gratis. Package-package tersebut disimpan pada [Python Package Index \(PyPI\)](https://pypi.org/), yaitu repositori resmi package Python.

Package dari PyPI dapat diinstal menggunakan program bernama pip yang telah tersedia secara bawaan pada instalasi Python.

Mengelola Package dengan pip

Menginstal Package dengan pip

Untuk menginstal sebuah package dari PyPI, gunakan perintah berikut pada Terminal atau *Command Prompt*.

```
pip install nama_package
```

Contoh menginstal package [numpy](https://pypi.org/project/numpy/):

```
pip install numpy
```

Hasilnya:

```
D:\ridwan-nurfauzi\belajar Python>pip install numpy
Collecting numpy
  Downloading numpy-2.5.1-cp314-cp314-win_amd64.whl.metadata (6.6 kB)
  Downloading numpy-2.5.1-cp314-cp314-win_amd64.whl (12.6 MB)
    12.6/12.6 MB 1.4 MB/s 0:00:09
Installing collected packages: numpy
  WARNING: The scripts f2py.exe and numpy-config.exe are installed in 'C:\Users\Kaka Ridwan\AppData\Local\Python\pythoncore-3.14-64\Scripts' which is not on PATH.
  Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
Successfully installed numpy-2.5.1
```

Melihat Daftar Package yang Terinstal

Untuk melihat seluruh package yang telah terinstal pada Python, gunakan perintah berikut:

```
pip list
```

Hasilnya:

```
D:\ridwan-nurfauzi\belajar Python>pip list
Package Version
-----
numpy    2.5.1
pip      25.3
```

Menghapus Package dengan pip

Package yang sudah tidak diperlukan dapat dihapus menggunakan perintah berikut:

```
pip uninstall nama_package
```

Contoh:

```
pip uninstall numpy
```

Hasilnya:

```
D:\ridwan-nurfauzi\belajar Python>pip uninstall numpy
Found existing installation: numpy 2.5.1
Uninstalling numpy-2.5.1:
  Would remove:
    c:\users\kaka ridwan\appdata\local\python\pythoncore-3.14-64\lib\site-packages\numpy-2.5.1.dist-info\*
    c:\users\kaka ridwan\appdata\local\python\pythoncore-3.14-64\lib\site-packages\numpy.libs\libscipy_openblas64_-b788215d9d47792bcba3a2e2a7114320.dll
    c:\users\kaka ridwan\appdata\local\python\pythoncore-3.14-64\lib\site-packages\numpy.libs\msvc140-a4c2229bd-c2a2a630acdc095b4d86008.dll
    c:\users\kaka ridwan\appdata\local\python\pythoncore-3.14-64\lib\site-packages\numpy\*
    c:\users\kaka ridwan\appdata\local\python\pythoncore-3.14-64\scripts\f2py.exe
    c:\users\kaka ridwan\appdata\local\python\pythoncore-3.14-64\scripts\numpy-config.exe
Proceed (Y/n)? y
  Successfully uninstalled numpy-2.5.1
```

Bab 10: Pengolahan File

File adalah sekumpulan data digital yang disimpan dalam media penyimpanan, seperti hard disk, SSD, atau flash drive. File dapat berisi berbagai jenis data, seperti teks, gambar, audio, video, maupun data lainnya.

Selama ini, program Python yang telah dibuat hanya menyimpan data sementara di dalam variabel. Kelemahannya, ketika program selesai dijalankan atau ditutup, seluruh data yang tersimpan di dalam variabel akan hilang.

Oleh karena itu, diperlukan pengolahan file agar program dapat menyimpan data secara permanen. Dengan demikian, data tetap tersedia meskipun program telah ditutup dan dapat digunakan kembali saat program dijalankan di lain waktu.

Mode Pengolahan File

Sebelum mempelajari cara mengolah file, kita perlu mengenal berbagai mode yang digunakan saat membuka file. Setiap mode memiliki fungsi yang berbeda, seperti membaca, menulis, atau menambahkan data ke dalam file.

Mode **r**

Mode **r** (*read*) digunakan untuk membuka dan membaca isi file. Jika file yang akan dibuka tidak ditemukan, Python akan menghasilkan error.

Mode **w**

Mode **w** (*write*) digunakan untuk menulis data ke dalam file. Jika file sudah ada, seluruh isi file akan dihapus dan diganti dengan data yang baru. Jika file belum ada, Python akan membuat file tersebut secara otomatis.

Mode **a**

Mode **a** (*append*) digunakan untuk menambahkan data ke bagian akhir file tanpa menghapus isi yang sudah ada. Jika file belum ada, Python akan membuat file tersebut secara otomatis.

Mode x

Mode x (*exclusive creation*) digunakan untuk membuat file baru dan menulis data ke dalamnya. Jika file dengan nama yang sama sudah ada, Python akan menghasilkan error sehingga file yang lama tidak akan tertimpa.

Mode rb

Mode rb (*read binary*) digunakan untuk membaca file dalam bentuk biner (*binary*). Mode ini biasanya digunakan untuk file nonteks, seperti gambar, audio, video, atau dokumen PDF.

Mode wb

Mode wb (*write binary*) digunakan untuk menulis data dalam bentuk biner. Sama seperti [mode w](#), jika file sudah ada, seluruh isinya akan dihapus terlebih dahulu. Mode ini umumnya digunakan untuk file nonteks.

Mode ab

Mode ab (*append binary*) digunakan untuk menambahkan data dalam bentuk biner ke bagian akhir file tanpa menghapus isi yang sudah ada.

Mode r+

Mode r+ digunakan untuk membuka file yang sudah ada agar dapat dibaca sekaligus ditulis atau diubah. Jika file tidak ditemukan, Python akan menghasilkan error.

Membuka dan Menutup File

Untuk mengolah file di Python, langkah pertama yang harus dilakukan adalah membuka file tersebut. Setelah selesai digunakan, file sebaiknya ditutup agar sumber daya sistem dapat dibebaskan dan perubahan pada file tersimpan dengan baik.

Python menyediakan fungsi `open()` untuk membuka file.

Bentuk dasar:

```
open(nama_file, mode)
```

Keterangan:

- `nama_file` adalah nama atau lokasi file yang akan dibuka.
- `mode` adalah [mode pengolahan file](#), seperti `r`, `w`, `a`, atau mode lainnya.

Contoh membuka file dalam mode baca (`r`):

```
file = open("data.txt", "r")
```

Pada contoh di atas, Python akan membuka file bernama `data.txt` dalam mode baca. Setelah file berhasil dibuka, objek file tersebut disimpan ke dalam variabel `file`.

Jika file `data.txt` tidak ada, maka akan muncul error seperti berikut.

```
FileNotFoundError: [Errno 2] No such file or directory: 'data.txt'
```

Setelah selesai menggunakan file, sebaiknya file ditutup menggunakan fungsi `close()`.

Bentuk dasar:

```
nama_variabel.close()
```

Contoh:

```
file = open("data.txt", "r")  
  
file.close()
```

Pada contoh di atas, file dibuka menggunakan fungsi `open()`, kemudian ditutup menggunakan fungsi `close()`. Menutup file merupakan kebiasaan yang baik karena dapat mengurangi penggunaan sumber daya sistem dan memastikan seluruh perubahan pada file telah disimpan dengan benar.

Menulis File

Untuk menulis data ke dalam file, kita dapat menggunakan [mode `w`](#) (*write*). Jika file belum ada, Python akan membuat file tersebut secara otomatis. Namun, jika file sudah ada, seluruh isi file akan dihapus dan diganti dengan data yang baru.

Setelah file berhasil dibuka, kita dapat menggunakan fungsi `write()` untuk menulis data ke dalam file.

Bentuk dasar:

```
nama_variabel.write(data)
```

Contoh:

```
file = open("data.txt", "w")

file.write("Halo, Python!")

file.close()
```

Pada contoh di atas, Python akan membuat file bernama `data.txt` (jika belum ada), kemudian menuliskan teks `"Halo, Python!"` ke dalam file tersebut. Setelah proses penulisan selesai, file ditutup menggunakan fungsi `close()`.

Jika file `data.txt` sebelumnya sudah memiliki isi, maka seluruh isi lama akan dihapus dan diganti dengan teks yang baru.

Fungsi `write()` hanya dapat digunakan untuk menulis data bertipe `string`. Jika ingin menulis tipe data lain, seperti `int` atau `float`, ubahlah terlebih dahulu menjadi `string` menggunakan fungsi `str()`.

Contoh:

```
file = open("data.txt", "w")

umur = 20

file.write(str(umur))

file.close()
```

Pada contoh di atas, nilai `20` diubah menjadi `string` menggunakan fungsi `str()`, sehingga dapat ditulis ke dalam file tanpa menghasilkan error.

Menulis Beberapa Baris

Karakter khusus `\n` (*newline*) dapat digunakan untuk membuat baris baru di dalam file.

Contoh:

```
file = open("data.txt", "w")

file.write("Baris pertama\n")
file.write("Baris kedua\n")
file.write("Baris ketiga")

file.close()
```

Hasil isi file data.txt:

```
Baris pertama
Baris kedua
Baris ketiga
```

Menulis Beberapa Baris dengan `writelines()`

Selain menggunakan `write()`, Python juga menyediakan fungsi `writelines()` untuk menulis beberapa data sekaligus ke dalam file. Fungsi ini menerima sebuah objek iterable, seperti `list` atau `tuple`, yang berisi sekumpulan string.

Contoh:

```
file = open("data.txt", "w")

baris = [
    "Baris pertama\n",
    "Baris kedua\n",
    "Baris ketiga\n"
]

file.writelines(baris)

file.close()
```

Hasil isi file data.txt:

```
Baris pertama
Baris kedua
Baris ketiga
```

Perlu diperhatikan bahwa fungsi `writelines()` tidak menambahkan karakter baris baru (`\n`) secara otomatis. Oleh karena itu, jika ingin setiap data ditulis pada baris yang berbeda, karakter `\n` harus ditambahkan secara manual.

Sebagai contoh, jika karakter `\n` tidak ditambahkan:

```
file = open("data.txt", "w")

baris = [
    "Baris pertama",
    "Baris kedua",
    "Baris ketiga"
]

file.writelines(baris)

file.close()
```

Maka isi file akan menjadi:

```
Baris pertamaBaris keduaBaris ketiga
```

Menambahkan Isi File

Selain menulis file menggunakan `mode w`, Python juga menyediakan `mode a` (*append*) untuk menambahkan data ke dalam file.

Perbedaan utama antara `mode w` dan `a` adalah `mode w` akan menghapus seluruh isi file sebelum menulis data baru, sedangkan `mode a` akan menambahkan data di bagian akhir file tanpa menghapus isi yang sudah ada. Jika file belum ada, Python akan membuat file tersebut secara otomatis.

Karena file dibuka dalam `mode a`, kita tetap dapat menggunakan fungsi `write()` maupun `writelines()` untuk menambahkan data.

Menambahkan Data dengan `write()`

Contoh:

```
file = open("data.txt", "a")
file.write("\nBaris ketiga")
file.close()
```

Misalkan isi awal file `data.txt` adalah:

```
Baris pertama  
Baris kedua
```

Setelah program dijalankan, isi file akan menjadi:

```
Baris pertama  
Baris kedua  
Baris ketiga
```

Pada contoh di atas, karakter `\n` digunakan agar data baru ditulis pada baris berikutnya. Tanpa karakter tersebut, data baru akan langsung menyambung dengan baris terakhir.

Menambahkan Beberapa Baris dengan `writelines()`

Fungsi `writelines()` juga dapat digunakan untuk menambahkan beberapa baris sekaligus ke dalam file.

Contoh:

```
file = open("data.txt", "a")  
  
baris = [  
    "\nBaris keempat",  
    "\nBaris kelima",  
    "\nBaris keenam"  
]  
  
file.writelines(baris)  
  
file.close()
```

Setelah program dijalankan, isi file akan menjadi:

```
Baris pertama  
Baris kedua  
Baris ketiga  
Baris keempat  
Baris kelima  
Baris keenam
```

Seperti halnya saat menulis file, fungsi `writelines()` tidak menambahkan karakter baris baru (`\n`) secara otomatis. Oleh karena itu, jika setiap data ingin ditulis pada baris yang berbeda, karakter `\n` harus ditambahkan secara manual.

Membaca File

Untuk membaca isi file, kita dapat membuka file menggunakan `mode r` (`read`). Setelah file berhasil dibuka, Python menyediakan beberapa `fungsi` untuk membaca isinya, yaitu `read()`, `readline()`, dan `readlines()`.

Jika file yang dibuka dengan `mode r` tidak ditemukan, Python akan menghasilkan error `FileNotFoundError`.

Membaca Seluruh Isi File dengan `read()`

`Fungsi read()` digunakan untuk membaca seluruh isi file sekaligus.

Contoh:

```
file = open("data.txt", "r")

isi = file.read()

print(isi)

file.close()
```

Misalkan isi file `data.txt` adalah:

```
Baris pertama
Baris kedua
```

Maka output program adalah:

```
Baris pertama
Baris kedua
```

Selain membaca seluruh isi file, `read()` juga dapat menerima sebuah bilangan bulat sebagai parameter untuk menentukan jumlah karakter yang akan dibaca.

Contoh:

```
file = open("data.txt", "r")

isi = file.read(5)

print(isi)

file.close()
```

Output:

```
Baris
```

Pada contoh di atas, hanya lima karakter pertama dari file yang dibaca.

Membaca Satu Baris dengan `readline()`

Fungsi `readline()` digunakan untuk membaca satu baris setiap kali dipanggil.

Contoh:

```
file = open("data.txt", "r")

print(file.readline())
print(file.readline())
print(file.readline())

file.close()
```

Misalkan isi file `data.txt` adalah:

```
Baris pertama
Baris kedua
Baris ketiga
```

Output:

```
Baris pertama

Baris kedua

Baris ketiga
```

Setiap pemanggilan `readline()` akan membaca satu baris berikutnya hingga mencapai akhir file.

Membaca Seluruh Baris dengan `readlines()`

Fungsi `readlines()` digunakan untuk membaca seluruh isi file dan mengembalikannya dalam bentuk `list`. Setiap elemen dalam `list` mewakili satu baris pada file.

Contoh:

```
file = open("data.txt", "r")

baris = file.readlines()

print(baris)

file.close()
```

Misalkan isi file `data.txt` adalah:

```
Baris pertama
Baris kedua
Baris ketiga
```

Output:

```
['Baris pertama\n', 'Baris kedua\n', 'Baris ketiga']
```

Karakter `\n` masih terlihat karena merupakan karakter pindah baris yang ikut tersimpan pada setiap baris, kecuali baris terakhir jika tidak diakhiri dengan karakter tersebut.

Karena `readlines()` mengembalikan sebuah `list`, kita dapat mengakses setiap elemennya menggunakan `indeks` atau melakukan `perulangan`.

Contoh:

```
file = open("data.txt", "r")

baris = file.readlines()

for data in baris:
    print(data, end="")

file.close()
```

Output:

```
Baris pertama  
Baris kedua  
Baris ketiga
```

Pada contoh di atas, argumen `end=""` digunakan agar fungsi `print()` tidak menambahkan karakter baris baru lagi, karena setiap elemen `list` sudah mengandung karakter `\n`.

Penggunaan `with open()`

Pada materi sebelumnya, file dibuka menggunakan fungsi `open()` dan ditutup secara manual menggunakan fungsi `close()`. Cara tersebut sudah benar, tetapi Python menyediakan cara yang lebih praktis, yaitu menggunakan pernyataan `with`.

Dengan `with`, file akan ditutup secara otomatis setelah selesai digunakan, sehingga kita tidak perlu lagi memanggil fungsi `close()`.

Bentuk dasar:

```
with open(nama_file, mode) as nama_variabel:  
    # proses pengolahan file
```

Contoh membaca file:

```
with open("data.txt", "r") as file:  
    isi = file.read()  
  
print(isi)
```

Contoh menulis file:

```
with open("data.txt", "w") as file:  
    file.write("Halo, Python!")
```

Karena file akan ditutup secara otomatis setelah keluar dari blok `with`, penggunaan cara ini lebih disarankan dibandingkan memanggil `open()` dan `close()` secara manual.